17 Oktober 2026

# ITFEST POLSRI 2026

**Presented by** The Team **BDCA - GUBERNUR HELIKOPTER**

**Ahmad Ghumaydi (Caom4y)**
**Agung Syaputra (Cleric)**
**Bagaskara Amukti Palapa (Mooncrust)**

bidarcyberarmy.org | Bina Darma Cyber Army | @bidarcyberarmy

Save For Later

**Digital Forensic** **3**
Selendang Sutra yang Terbakar 3
Bisikan Angin Utara 5
Kopi Pahit Pukul Tiga 7
Bunga Kering di Antara Halaman 9
Jejak Roda di Lumpur 11
Perjamuan Malam Tanpa Tamu 13
Lilin Leleh di Meja Makan 15
Perahu Kertas di Arus Deras 17
Ranting Patah Menunjuk Arah 19
Topeng Kayu dari Seberang 21
**Reverse Engineering** **23**
Saksi Bisu Lorong Gelap 23
Tarian Debu di Sela Jendela 25
Arloji Perak yang Berhenti 27
Catatan Pinggir Sang Pengembara 29
Cermin Tua di Loteng 32
Melodi Hujan Bulan Oktober 33
Patung Batu di Dasar Danau 35
Potret Keluarga Tahun Delapan Puluh 37
Senja di Pelabuhan Tua 39
Sepasang Sepatu Usang 41
**Web Exploitation** **45**
Gerbang Penjara Jiwa 45
Kotak Peti Keramat 47
Celah Tirai Panggung 50
Karcis Singgasana Maharaja 51
Topeng Wajah Bayangan 53
Kurir Utusan Rahasia 56
Kitab Larangan Leluhur 58
Sumpah Serapah Angka Nol 60
Labirin Gulungan Sandi 63
Sumur Tanpa Dasar 65
**Cryptography** **68**
Gema dari Dasar Sumur 68
Kunci Karatan di Saku Jas 69
Langkah Kaki di Atas Daun 71
Teka Teki Kucing Hitam 73
Surat yang Tak Pernah Sampai 75
Parade Bayangan Semu 76
benang Merah yang Kusut 78
Cangkir Keramik Retak 80
Nyanyian Burung Besi 82
Peta Menuju Pulau Khayalan 84

# Digital Forensic

## Selendang Sutra yang Terbakar

| | |
|---|---|
| **Deskripsi** | Di tengah lautan lumpur yang penuh dengan seribu botol kaca berisi pesan palsu untuk mengecoh para penyelam serakah, terdapat satu kantung udara yang dipadatkan dengan tekanan tinggi. Hanya penyelam yang mampu mengenali kepala kantung bertekanan tinggi dan memompanya kembali yang dapat membaca pesan sejatinya. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/dc46da49bfb378fff45cf1aaa3235b84/forensic_2_dasar_lautan.dat |
| **Flag** | ITFest2026{g3l3mbung_ud4r4_d1_d4s4r_l4ut4n_b1s1ng_22b3} |

Petunjuk tentang kantung udara yang dipadatkan mengarah pada data terkompresi. Pesan yang terlihat langsung dalam file perlu diperiksa karena deskripsi menyebut adanya pesan palsu.

```
~/Documents/CTF/DIGFOR/Selendang Sutra yang Terbakar
❯ file forensic_2_dasar_lautan.dat
forensic_2_dasar_lautan.dat: data
```

File berukuran 945 byte dan terdeteksi sebagai data. Di dalamnya terdapat pengulangan pesan pengecoh:

```
BOTOL_PALSU: ITFest2026{jangan_ambil_botol_lumpur_ini}
BOTOL_PALSU: ITFest2026{jangan_ambil_botol_lumpur_ini}
BOTOL_PALSU: ITFest2026{jangan_ambil_botol_lumpur_ini}
BOTOL_PALSU: ITFest2026{jangan_ambil_botol_lumpur_ini}
```

Selain pesan tersebut, ada penanda yang membatasi sebuah payload biner:

```
KANTUNG_UDARA_START>>x
qK-.1202
/M1)2
01)6)
1)-1
02,6
Q<<KANTUNG_UDARA_END
```

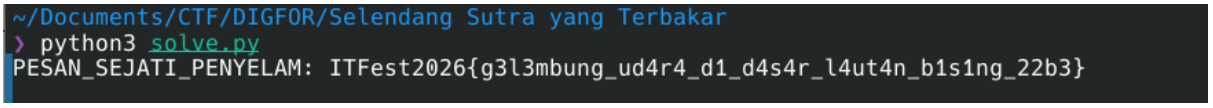
Payload dimulai pada offset desimal 681, atau 0x2A9, dan panjangnya 85 byte. Dua byte pertamanya adalah 78 DA, header zlib yang sesuai dengan petunjuk kompresi. Payload ini dapat dipulihkan menggunakan zlib.decompress().

```
from pathlib import Path
import zlib

data = Path("forensic_2_dasar_lautan.dat").read_bytes()
start_marker = b"KANTUNG_UDARA_START>>"
end_marker = b"<<KANTUNG_UDARA_END"

start = data.index(start_marker) + len(start_marker)
end = data.index(end_marker, start)
payload = data[start:end]


print(zlib.decompress(payload).decode())
```




**ITFest2026{g3l3mbung_ud4r4_d1_d4s4r_l4ut4n_b1s1ng_22b3}**

# Bisikan Angin Utara

| | |
|---|---|
| **Deskripsi** | Sebuah boneka kayu dari negeri utara ditemukan di atas meja. Saat dibuka lapisan luarnya yang terdiri dari enam belas warna, terlihat boneka kedua milik Kompi 32. Di dalam perut Kompi 32, terdapat kantung udara yang dipadatkan dengan sangat kencang, yang membungkus boneka terkecil milik Kompi 64. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/bbb4a37d4acd95a659681bc434db4d9c/forensic_7_boneka_utara.txt |
| **Flag** | ITFest2026{b0n3k4_k4yu_b3rl4p1s_3mp4t_ru4ng_77a8} |

Boneka berlapis dalam deskripsi menggambarkan data yang dibungkus beberapa kali. Petunjuknya memberikan urutan pembongkaran berikut:

| **Petunjuk** | **Operasi** |
|---|---|
| Enam belas warna | Decode hexadecimal atau Base16 |
| Kompi 32 | Decode Base32 |
| Kantung udara dipadatkan | Dekompresi |
| Kompi 64 | Decode Base64 |

Isi file berupa karakter hexadecimal. Setelah di-decode, hasilnya adalah string Base32:

```
PCOAWDQLOKHYV4BT6DGXFLHUZMV4T5DVG6UUVLAKJIBWFU4UGDFUYX3DV5RD6D6HRJSE
HC5KUQEEPAYUG7F4VFEQUDJCQQYLMM7SGV4TWRAFWWYACK2BMCA=
```

Decode Base32 menghasilkan data biner dengan dua byte awal 78 9C, yaitu header zlib. Setelah didekompresi menggunakan zlib, diperoleh string Base64:

```
SVRGZXN0MjAyNntiMG4zazRfazR5dV9iM3JsNHAxc18zbXA0dF9ydTRuZ183N2E4fQ==
```

Decode Base64 pada lapisan terakhir menghasilkan flag dalam bentuk teks.

```
from pathlib import Path
import base64
import zlib

encoded = Path("forensic_7_boneka_utara.txt").read_text().strip()
layer1 = bytes.fromhex(encoded)
layer2 = base64.b32decode(layer1)
layer3 = zlib.decompress(layer2)
flag = base64.b64decode(layer3).decode()
```

print(flag)

```
~/Documents/CTF/DIGFOR/Bisikan Angin Utara
❯ python3 solve.py
ITFest2026{b0n3k4_k4yu_b3rl4p1s_3mp4t_ru4ng_77a8}
```

**ITFest2026{b0n3k4_k4yu_b3rl4p1s_3mp4t_ru4ng_77a8}**

# Kopi Pahit Pukul Tiga

| | |
|---|---|
| **Deskripsi** | Angin puyuh merobek buku catatan itu menjadi puluhan cabikan kecil dan mengacak urutannya di seluruh halaman. Untungnya, sebelum buku itu robek, sang penulis telah menuliskan nomor halaman kecil di sudut kiri setiap cabikan. Susun kembali cabikan yang memiliki segel 'ASLI' sesuai urutan nomornya. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/8a08851607ccd874fa5fc0bc6836d02b/forensic_9_cabikan_angin.txt |
| **Flag** | ITFest2026{p0t0ng4n_sur4t_y4ng_d1t1up_4ng1n_puyuh_99c0} |

File berisi 28 baris dengan struktur berikut:

```
~/Documents/CTF/DIGFOR/Kopi Pahit Pukul Tiga
❯ cat forensic_9_cabikan_angin.txt
HAL:12|SEGEL:ASLI|ISI:aF85OQ==
HAL:02|SEGEL:PALSU|ISI:WFhYWA==
HAL:04|SEGEL:ASLI|ISI:ZzRuXw==
HAL:13|SEGEL:PALSU|ISI:WFhYWA==
HAL:08|SEGEL:ASLI|ISI:MXQxdQ==
HAL:12|SEGEL:PALSU|ISI:WFhYWA==
HAL:11|SEGEL:ASLI|ISI:cHV5dQ==
HAL:05|SEGEL:ASLI|ISI:c3VyNA==
HAL:09|SEGEL:ASLI|ISI:cF80bg==
```

Setiap baris memiliki tiga field:

| Field | Fungsi |
|---|---|
| HAL | Nomor urutan potongan |
| SEGEL | Penanda potongan asli atau palsu |
| ISI | Isi potongan yang di-encode dengan Base64 |

Sesuai deskripsi, hanya baris dengan SEGEL:ASLI yang digunakan. Setelah dipilih, terdapat 14 potongan bernomor 00 sampai 13. Urutkan berdasarkan nilai angka HAL, decode Base64 setiap ISI, lalu gabungkan hasilnya.

Setiap potongan harus di-decode secara terpisah karena masing-masing memiliki padding Base64 sendiri.

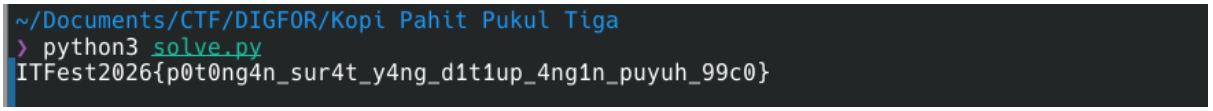
```
from pathlib import Path
import base64

lines = Path("forensic_9_cabikan_angin.txt").read_text().splitlines()
parts = []
```

```
for line in lines:
   row = dict(field.split(":", 1) for field in line.split("|"))
   if row["SEGEL"] == "ASLI":
      number = int(row["HAL"])
      content = base64.b64decode(row["ISI"])
      parts.append((number, content))

parts.sort(key=lambda item: item[0])
flag = b"".join(content for _, content in parts).decode()

print(flag)
```




**ITFest2026{p0t0ng4n_sur4t_y4ng_d1t1up_4ng1n_puyuh_99c0}**

# Bunga Kering di Antara Halaman

| | |
|---|---|
| **Deskripsi** | Bendahara kota berusaha menghapus satu baris transaksi gelap di buku besarnya dengan cara menandai status baris tersebut menjadi angka nol (0x00), sementara ratusan baris transaksi palsu lainnya ditandai dengan angka satu (0x01). Carilah satu-satunya baris yang telah dicoret tersebut lalu kurangi setiap angkanya dengan garam bernilai 17. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/71cd41ea7dbfcf8264b1556a20664871/forensic_10_buku_bendahara.dat |
| **Flag** | ITFest2026{c4t4t4n_y4ng_d1c0r3t_t4p1_m4s1h_t3rb4c4_00d1} |

Berkas merupakan teks ASCII berukuran 2.417 byte. Baris pertama berisi penanda format:

```
~/Documents/CTF/DIGFOR/Bunga Kering di Antara Halaman
❯ cat forensic_10_buku_bendahara.dat
BUKU_BESAR_KOTA_V1
01002c5a6557768485434143478c8583727f84727c847a7081727d8486707f807e8083704141707273727a7c727f
8e
01002c5a6557768485434143478c8583727f84727c847a7081727d8486707f807e8083704142707273727a7c727f
8e
01002c5a6557768485434143478c8583727f84727c847a7081727d8486707f807e8083704143707273727a7c727f
8e
01002c5a6557768485434143478c8583727f84727c847a7081727d8486707f807e8083704144707273727a7c727f
8e
01002c5a6557768485434143478c8583727f84727c847a7081727d8486707f807e8083704145707273727a7c727f
8e
```

Baris berikutnya berisi record dalam representasi heksadesimal. Karena itu, pencarian byte mentah 0x00 pada berkas tidak akan menemukan status transaksi: teks hex harus dikonversi menjadi byte terlebih dahulu.

Setelah dikonversi, struktur setiap record adalah:

| Offset | Ukuran | Isi |
|---|---|---|
| 0 | 1 byte | Status: 0x01 atau 0x00 |
| 1–2 | 2 byte | Panjang payload, big-endian |
| 3 dan seterusnya | Sesuai panjang | Payload yang telah ditambah 17 |

Interpretasi panjang tersebut cocok dengan ukuran payload seluruh record. Berkas aktual memuat 25 record: 24 berstatus 0x01 dan satu berstatus 0x00.

Record target berada pada baris ke-16 berkas, atau record ke-15 jika header tidak dihitung. Tiga byte awalnya adalah:

```
00 00 38
```

00 pertama merupakan status dihapus. Dua byte berikutnya, 00 38, menyatakan panjang payload 0x0038 = 56 byte.

Petunjuk “kurangi setiap angkanya dengan garam bernilai 17” diterapkan pada byte payload menggunakan pengurangan 17 desimal (0x11). Misalnya, byte payload pertama 0x5A dikurangi 0x11 menghasilkan 0x49, yaitu karakter I.

```
from pathlib import Path

source = Path("forensic_10_buku_bendahara.dat")
lines = source.read_text(encoding="ascii").splitlines()
assert lines[0] == "BUKU_BESAR_KOTA_V1"

deleted_records = []
for line_number, line in enumerate(lines[1:], start=2):
    record = bytes.fromhex(line)
    assert len(record) >= 3, "Header record tidak lengkap"

    status = record[0]
    length = int.from_bytes(record[1:3], byteorder="big")
    payload = record[3:]
    assert len(payload) == length, "Panjang payload tidak sesuai"

    if status == 0x00:
        deleted_records.append((line_number, payload))

assert len(deleted_records) == 1, "Jumlah record terhapus tidak sesuai"
line_number, payload = deleted_records[0]
decoded = bytes((value - 17) % 256 for value in payload)

Path("transaksi_dihapus.txt").write_bytes(decoded)
print("Baris:", line_number)
print(decoded.decode("ascii"))
```

```
~/Documents/CTF/DIGFOR/Bunga Kering di Antara Halaman
❯ python3 solve.py
Baris: 16
ITFest2026{c4t4t4n_y4ng_d1c0r3t_t4p1_m4s1h_t3rb4c4_00d1}
```

**ITFest2026{c4t4t4n_y4ng_d1c0r3t_t4p1_m4s1h_t3rb4c4_00d1}**

# Jejak Roda di Lumpur

| | |
|---|---|
| **Deskripsi** | Tuan rumah mengadakan jamuan besar dengan ratusan piring makanan. Para tamu sibuk menyantap hidangan utama, padahal pesan rahasia justru disusun dari satu remah terkecil yang tersisa di bagian paling ujung kanan dari setiap piring. Setiap 8 remah yang dikumpulkan akan membentuk satu huruf utuh. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/2c54c7e8b7782f6ff71df559f0c87abc/forensic_8_remah_jamuan.bin |
| **Flag** | ITFest2026{r3m4h_r0t1_p4l1ng_k3c1l_d1_ujung_m3j4_88b9} |

Petunjuk “satu remah terkecil” di “paling ujung kanan” mengarah pada Least Significant Bit (LSB), yaitu bit paling kanan dari setiap byte. Petunjuk berikutnya menjelaskan bahwa setiap delapan bit membentuk satu karakter.

Berkas berukuran 432 byte. Jika setiap byte menyumbang satu bit, panjang pesan hasil ekstraksi adalah:

```
432 bit / 8 = 54 byte
```

LSB dapat diambil dengan operasi value & 1. Urutan byte sumber dipertahankan, lalu setiap kelompok delapan bit dibaca dengan bit pertama sebagai bit paling signifikan karakter hasil.

Sebagai contoh, delapan byte pertama adalah:

```
Byte: 54 53 44 58 57 4C 40 4B
LSB :  0  1  0  0  1  0  0  1
```

Bit 01001001 bernilai 0x49, yaitu karakter I.

```
from pathlib import Path

source = Path("forensic_8_remah_jamuan.bin")
data = source.read_bytes()

bits = "".join(str(value & 1) for value in data)
assert len(bits) % 8 == 0, "Jumlah bit bukan kelipatan delapan"

decoded = bytes(
    int(bits[offset:offset + 8], 2)
    for offset in range(0, len(bits), 8)
)

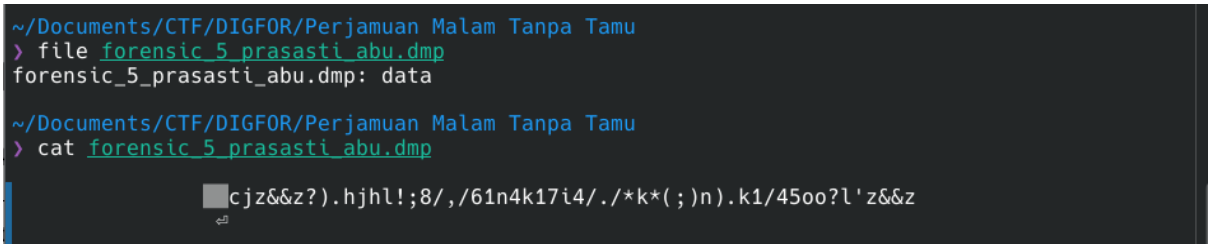
Path("remah_decoded.txt").write_bytes(decoded)
print(decoded.decode("ascii"))
```

```
~/Documents/CTF/DIGFOR/Jejak Roda di Lumpur
❯ python3 solve.py
ITFest2026{r3m4h_r0t1_p4l1ng_k3c1l_d1_ujung_m3j4_88b9}
```

**ITFest2026{r3m4h_r0t1_p4l1ng_k3c1l_d1_ujung_m3j4_88b9}**

# Perjamuan Malam Tanpa Tamu

| | |
|---|---|
| **Deskripsi** | Letusan gunung berapi telah menyelimuti seluruh permukaan prasasti batu itu dengan lapisan abu seragam setebal 90 milimeter (0x5A) di setiap jengkalnya. Bersihkan lapisan abu yang seragam itu terlebih dahulu sebelum mencari kalimat yang terukir di tengah batu. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/f8195506c3858d30c8bf346f21f1aa6d/forensic_5_prasasti_abu.dmp |
| **Flag** | ITFest2026{abu_vulk4n1k_m3nutup1_pras4st1_kuno_55e6} |




Berkas berukuran 101 byte dan terdeteksi sebagai data biner. Petunjuk menyebut nilai konstan 90, atau 0x5A, yang diterapkan merata pada seluruh prasasti. Ini mengarah pada percobaan XOR setiap byte dengan kunci 0x5A.

XOR bersifat reversibel. Jika byte asli disamarkan dengan XOR, operasi yang sama mengembalikannya:

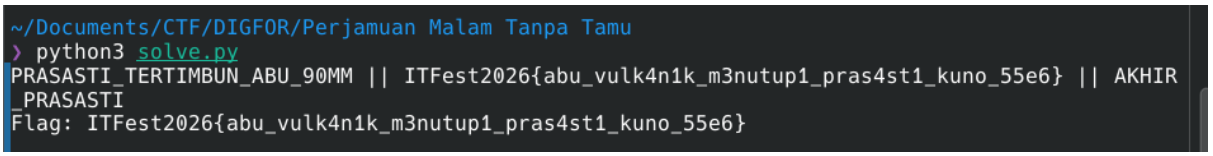
```
encoded = original XOR 0x5A
original = encoded XOR 0x5A
```

Byte pertama berkas adalah 0x0A. Setelah di-XOR dengan 0x5A, hasilnya 0x50, yaitu karakter P. Penerapan pada seluruh berkas menghasilkan teks yang terbaca.

```
from pathlib import Path
import re

source = Path("forensic_5_prasasti_abu.dmp")
decoded = bytes(value ^ 0x5A for value in source.read_bytes())
Path("prasasti_bersih.txt").write_bytes(decoded)

print(decoded.decode("ascii"))
flag = re.search(rb"ITFest2026\{[^}]+\}", decoded)
assert flag is not None, "Flag tidak ditemukan"
print("Flag:", flag.group().decode("ascii"))
```

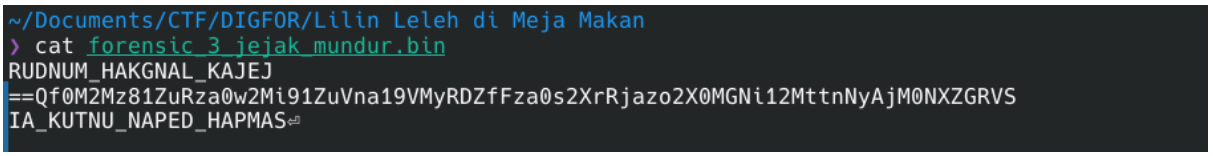



**ITFest2026{abu_vulk4n1k_m3nutup1_pras4st1_kuno_55e6}**

# Lilin Leleh di Meja Makan

| | |
|---|---|
| **Deskripsi** | Seseorang berjalan mundur dari puncak gunung hingga ke lembah sambil menuliskan kisahnya huruf demi huruf, bit demi bit, dari langkah terakhir menuju langkah pertama. Jika kamu menatapnya secara normal, kamu hanya akan melihat tumpukan sampah tak bermakna. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/0d6342d32378f8c49219cd4e36ba5b42/forensic_3_jejak_mundur.bin |
| **Flag** | ITFest2026{m3mb4c4_j3j4k_k4k1_d4r1_ujung_b3l4k4ng_33c4} |

File berukuran 119 byte. Walaupun berekstensi .bin, isinya berupa teks. Salah satu barisnya adalah RUDNUM_HAKGNAL_KAJEJ, yang menjadi JEJAK_LANGKAH_MUNDUR jika dibaca terbalik.




Petunjuk berjalan mundur mengarah pada pembalikan urutan byte seluruh file.

Setelah urutan byte dibalik, isi file menjadi:

```
SAMPAH_DEPAN_UNTUK_AI
SVRGZXN0MjAyNnttM21iNGM0X2ozajRrX2s0azFfZDRyMV91anVuZ19iM2w0azRuZ18zM2M0fQ==

JEJAK_LANGKAH_MUNDUR
```

Baris pertama dan terakhir adalah penanda. Baris kedua merupakan payload Base64. Decode baris tersebut untuk mendapatkan flag. Tidak diperlukan pembalikan bit di dalam setiap byte.

```
from pathlib import Path
import base64

data = Path("forensic_3_jejak_mundur.bin").read_bytes()
restored = data[::-1]
payload = restored.splitlines()[1]
flag = base64.b64decode(payload, validate=True).decode()

print(flag)
```

```
~/Documents/CTF/DIGFOR/Lilin Leleh di Meja Makan
❯ file forensic_3_jejak_mundur.bin
forensic_3_jejak_mundur.bin: ASCII text

~/Documents/CTF/DIGFOR/Lilin Leleh di Meja Makan
❯ python3 solve.py
ITFest2026{m3mb4c4_j3j4k_k4k1_d4r1_ujung_b3l4k4ng_33c4}
```

**ITFest2026{m3mb4c4_j3j4k_k4k1_d4r1_ujung_b3l4k4ng_33c4}**

# Perahu Kertas di Arus Deras

| | |
|---|---|
| **Deskripsi** | Sebuah lukisan kaca ditemukan pecah tepat di bagian mahkota kepalanya sehingga tak seorang pun bisa mengenali wajahnya. Namun sang pelukis sempat menyelipkan selembar catatan kecil tepat di balik punggung kanvas paling belakang setelah lukisan itu selesai ditutup rapat. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/f3469ad0e444028d985b7c0fc974476a/forensic_1_lukisan_retak.bin |
| **Flag** | ITFest2026{k3p4l4_luk1s4n_rus4k_d4n_3k0r_r4h4s14_11a2} |

File berukuran 409 byte. Delapan byte awalnya adalah:

```
~/Documents/CTF/DIGFOR/Perahu Kertas di Arus Deras
❯ xxd forensic_1_lukisan_retak.bin
00000000: 0000 4e47 0d0a 1a0a 4445 434f 595f 464c  ..NG....DECOY_FL
00000010: 4147 3a20 4954 4665 7374 3230 3236 7b69  AG: ITFest2026{i
00000020: 6e69 5f68 616e 7961 5f75 6d70 616e 5f70  ni_hanya_umpan_p
00000030: 6572 6d75 6b61 616e 5f6b 6163 617d 0a44  ermukaan_kaca}.D
00000040: 4543 4f59 5f46 4c41 473a 2049 5446 6573  ECOY_FLAG: ITFes
00000050: 7432 3032 367b 696e 695f 6861 6e79 615f  t2026{ini_hanya_
00000060: 756d 7061 6e5f 7065 726d 756b 6161 6e5f  umpan_permukaan_
00000070: 6b61 6361 7d0a 4445 434f 595f 464c 4147  kaca}.DECOY_FLAG
00000080: 3a20 4954 4665 7374 3230 3236 7b69 6e69  : ITFest2026{ini
00000090: 5f68 616e 7961 5f75 6d70 616e 5f70 6572  _hanya_umpan_per
000000a0: 6d75 6b61 616e 5f6b 6163 617d 0a44 4543  mukaan_kaca}.DEC
000000b0: 4f59 5f46 4c41 473a 2049 5446 6573 7432  OY_FLAG: ITFest2
000000c0: 3032 367b 696e 695f 6861 6e79 615f 756d  026{ini_hanya_um
000000d0: 7061 6e5f 7065 726d 756b 6161 6e5f 6b61  pan_permukaan_ka
000000e0: 6361 7d0a 4445 434f 595f 464c 4147 3a20  ca}.DECOY_FLAG:
000000f0: 4954 4665 7374 3230 3236 7b69 6e69 5f68  ITFest2026{ini_h
00000100: 616e 7961 5f75 6d70 616e 5f70 6572 6d75  anya_umpan_permu
00000110: 6b61 616e 5f6b 6163 617d 0a49 454e 44ae  kaan_kaca}.IEND.
00000120: 4260 820a 4341 5441 5441 4e5f 4241 4c49  B`..CATATAN_BALI
00000130: 4b5f 4b41 4e56 4153 3a55 315a 5352 3170  K_KANVAS:U1ZSR1p
00000140: 5954 6a42 4e61 6b46 3554 6d35 3063 6b30  YTjBNakF5Tm50ck0
00000150: 7a51 5442 6952 464a 6d59 6b68 5763 6b31  zQTBiRFJmYkhWck1
00000160: 5954 5442 6962 446c 355a 4668 4e4d 4745  YTTBibDl5ZFhNMGE
00000170: 784f 5774 4f52 7a56 6d54 544a 7a64 324e  xOWtORzVmTTJzd2N
00000180: 734f 586c 4f52 3263 7759 3370 464d 4668  sOXlOR2cwY3pFMFh
00000190: 3652 5868 5a56 456f 35                   6RXhZVEo5
```

| |
|---|
| 00 00 4E 47 0D 0A 1A 0A |

Bagian ini menyerupai signature PNG, tetapi dua byte pertamanya rusak. Signature PNG yang normal adalah:

| |
|---|
| 89 50 4E 47 0D 0A 1A 0A |

Petunjuk tentang kepala lukisan yang pecah sesuai dengan signature tersebut. Namun, isi file juga memuat pesan DECOY_FLAG berulang, sehingga kita perlu mencari payload lain sesuai petunjuk catatan di bagian belakang.

Di bagian akhir terdapat penanda IEND beserta byte AE 42 60 82, diikuti catatan:

```
CATATAN_BALIK_KANVAS:U1ZSR1pYTjBNakF5Tm50ck0zQTBiRFJmYkhWck1YTTBibDl5ZFhNMGExOWtORzVmTTJzd2NsOXlOR2cwY3pFMFh6RXhZVEo5
```

Dalam PNG normal, IEND menandai chunk terakhir. Di soal ini, informasi penting justru diletakkan setelah penanda tersebut. Kita dapat mengekstrak catatan langsung tanpa memperbaiki atau merender gambar.

Decode Base64 pertama menghasilkan string Base64 lain:

```
SVRGZXN0MjAyNntrM3A0bDRfbHVrMXM0bl9ydXM0a19kNG5fM2swcl9yNGg0czE0XzExYTJ9
```

Decode sekali lagi menghasilkan flag asli.

```
from pathlib import Path
import base64

data = Path("forensic_1_lukisan_retak.bin").read_bytes()
payload = data.split(b"CATATAN_BALIK_KANVAS:", 1)[1].strip()
layer1 = base64.b64decode(payload, validate=True)
flag = base64.b64decode(layer1, validate=True).decode()

print(flag)
```

```
~/Documents/CTF/DIGFOR/Perahu Kertas di Arus Deras
❯ python3 solve.py
ITFest2026{k3p4l4_luk1s4n_rus4k_d4n_3k0r_r4h4s14_11a2}
```

**ITFest2026{k3p4l4_luk1s4n_rus4k_d4n_3k0r_r4h4s14_11a2}**

# Ranting Patah Menunjuk Arah

| | |
|---|---|
| **Deskripsi** | Ribuan prajurit berbaris dalam satu antrean panjang. Sebagian besar dari mereka adalah pembohong yang meneriakkan kalimat tipuan. Sang jenderal berpesan bahwa hanya prajurit yang berdiri tepat di urutan bilangan prima (mulai dari urutan ke-2, 3, 5, 7, dan seterusnya) yang memegang potongan peta asli setelah digabung dan diputar oleh Kompi 64. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/57baf0648693f2b0dd88a735a0fa2e3c/forensic_6_barisan_prajurit.txt |
| **Flag** | ITFest2026{h4ny4_b1l4ng4n_pr1m4_y4ng_b3rk4t4_jujur_66f7} |

File berisi satu string sepanjang 384 karakter. Petunjuk prajurit pada urutan prima mengarah pada pemilihan karakter berdasarkan posisinya, sedangkan Kompi 64 mengarah pada Base64.

```
~/Documents/CTF/DIGFOR/Ranting Patah Menunjuk Arah
❯ cat forensic_6_barisan_prajurit.txt
C0SV0R4GCD2Z4X95CNA0C93M338B9j0A48DF6yCDDN4n052tDD95Fo85F74N4G3C0AB55FB50NED406F8F1906601iA1
EC7F9MA44WBw62600b98Fm2B615611EAC30c20F0C201Cb3lA9E2E268F99w6CD25c6C0D5j1B7F2AFEBtCDC17N2F7C
43DF92092526FN7G5AE0A73E6F85837FA744120nE4CX02159I79C10z9c6613061EEm00838sAE5B90AACEFd9D15DD
ARB2BfEa1FDA75CC4nB7BBD1B25537BV2DDq6d20FXC2CF996634E06JF320EfBCF7C03A2N8jC4BZ684C1m7A577F3N
FC23839033A0487=⏎
```

Ada detail indeks yang perlu diperhatikan: meskipun deskripsi menyebut urutan ke-2, ke-3, dan seterusnya, payload yang valid diperoleh dengan memakai indeks mulai dari nol. Artinya, indeks 2, 3, 5, dan 7 merujuk pada karakter ke-3, ke-4, ke-6, dan ke-8 dalam pembacaan biasa.

```
~/Documents/CTF/DIGFOR/Ranting Patah Menunjuk Arah
❯ xxd forensic_6_barisan_prajurit.txt
00000000: 4330 5356 3052 3447 4344 325a 3458 3935  C0SV0R4GCD2Z4X95
00000010: 434e 4130 4339 334d 3333 3842 396a 3041  CNA0C93M338B9j0A
00000020: 3438 4446 3679 4344 444e 346e 3035 3274  48DF6yCDDN4n052t
00000030: 4444 3935 466f 3835 4637 344e 3447 3343  DD95Fo85F74N4G3C
00000040: 3041 4235 3546 4235 304e 4544 3430 3646  0AB55FB50NED406F
00000050: 3846 3139 3036 3630 3169 4131 4543 3746  8F1906601iA1EC7F
00000060: 394d 4134 3457 4277 3632 3630 3062 3938  9MA44WBw62600b98
00000070: 466d 3242 3631 3536 3131 4541 4333 3063  Fm2B615611EAC30c
00000080: 3230 4630 4332 3031 4362 336c 4139 4532  20F0C201Cb3lA9E2
00000090: 4532 3638 4639 3977 3643 4432 3563 3643  E268F99w6CD25c6C
000000a0: 3044 356a 3142 3746 3241 4645 4274 4344  0D5j1B7F2AFEBtCD
000000b0: 4331 374e 3246 3743 3433 4446 3932 3039  C17N2F7C43DF9209
000000c0: 3235 3236 464e 3747 3541 4530 4137 3345  2526FN7G5AE0A73E
000000d0: 3646 3835 3833 3746 4137 3434 3132 306e  6F85837FA744120n
000000e0: 4534 4358 3032 3135 3949 3739 4331 307a  E4CX02159I79C10z
000000f0: 3963 3636 3133 3036 3145 456d 3030 3833  9c6613061EEm0083
00000100: 3873 4145 3542 3930 4141 4345 4664 3944  8sAE5B90AACEFd9D
00000110: 3135 4444 4152 4232 4266 4561 3146 4441  15DDARB2BfEa1FDA
00000120: 3735 4343 346e 4237 4242 4431 4232 3535  75CC4nB7BBD1B255
00000130: 3337 4256 3244 4471 3664 3230 4658 4332  37BV2DDq6d20FXC2
00000140: 4346 3939 3636 3334 4530 364a 4633 3230  CF996634E06JF320
00000150: 4566 4243 4637 4330 3341 324e 386a 4334  EfBCF7C03A2N8jC4
00000160: 425a 3638 3443 316d 3741 3537 3746 334e  BZ684C1m7A577F3N
00000170: 4643 3233 3833 3930 3333 4130 3438 373d  FC23839033A0487=
```

Percobaan menggunakan posisi mulai dari satu tidak menghasilkan teks UTF-8 yang valid setelah decode. Pemilihan indeks prima mulai dari nol menghasilkan string Base64 berikut:

```
SVRGZXN0MjAyNntoNG55NF9iMWw0bmc0bl9wcjFtNF95NG5nX2Izcms0dDRfanVqdXJfNjZmN30
=
```

String ini kemudian di-decode dengan Base64 untuk mendapatkan flag.

```
from pathlib import Path
from math import isqrt
import base64

def is_prime(n):
    if n < 2:
        return False
    return all(n % divisor != 0 for divisor in range(2, isqrt(n) + 1))

data = Path("forensic_6_barisan_prajurit.txt").read_text().strip()
payload = "".join(
    char for index, char in enumerate(data) if is_prime(index)
)
flag = base64.b64decode(payload, validate=True).decode()

print(flag)
```

```
~/Documents/CTF/DIGFOR/Ranting Patah Menunjuk Arah
❯ python3 solve.py
ITFest2026{h4ny4_b1l4ng4n_pr1m4_y4ng_b3rk4t4_jujur_66f7}
```

**ITFest2026{h4ny4_b1l4ng4n_pr1m4_y4ng_b3rk4t4_jujur_66f7}**

# Topeng Kayu dari Seberang

| | |
|---|---|
| **Deskripsi** | Buku catatan penjaga palang pintu kereta api itu tampak sangat membosankan. Ratusan kereta lewat membawa muatan batu bara yang sama persis. Namun para mata-mata tidak melihat muatan keretanya, melainkan memperhatikan angka milidetik tepat saat kereta berlabel 'MERPATI' melintasi garis perbatasan. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/fb35f65c43217d4afb6f52004a9104c2/forensic_4_jadwal_kereta.log |
| **Flag** | ITFest2026{w4ktu_k3d4t4ng4n_k3r3t4_y4ng_b3rb1s1k_44d5} |

File log berukuran 6.695 byte dan berisi 108 baris, terdiri dari 54 baris berlabel BATUBARA dan 54 baris berlabel MERPATI. Contoh awalnya:

```
2026-10-16 08:00:12.791 [BATUBARA] Muatan normal lewat
2026-10-16 08:00:15.073 [MERPATI] Kereta khusus melintasi perbatasan
2026-10-16 08:01:12.379 [BATUBARA] Muatan normal lewat
2026-10-16 08:01:15.084 [MERPATI] Kereta khusus melintasi perbatasan
```

```
~/Documents/CTF/DIGFOR/Topeng Kayu dari Seberang
❯ file forensic_4_jadwal_kereta.log

forensic_4_jadwal_kereta.log: ASCII text

~/Documents/CTF/DIGFOR/Topeng Kayu dari Seberang
❯ xxd forensic_4_jadwal_kereta.log
00000000: 3230 3236 2d31 302d 3136 2030 383a 3030  2026-10-16 08:00
00000010: 3a31 322e 3739 3120 5b42 4154 5542 4152  :12.791 [BATUBAR
00000020: 415d 204d 7561 7461 6e20 6e6f 726d 616c  A] Muatan normal
00000030: 206c 6577 6174 0a32 3032 362d 3130 2d31   lewat.2026-10-1
00000040: 3620 3038 3a30 303a 3135 2e30 3733 205b  6 08:00:15.073 [
00000050: 4d45 5250 4154 495d 204b 6572 6574 6120  MERPATI] Kereta
00000060: 6b68 7573 7573 206d 656c 696e 7461 7369  khusus melintasi
00000070: 2070 6572 6261 7461 7361 6e0a 3230 3236   perbatasan.2026
00000080: 2d31 302d 3136 2030 383a 3031 3a31 322e  -10-16 08:01:12.
00000090: 3337 3920 5b42 4154 5542 4152 415d 204d  379 [BATUBARA] M
000000a0: 7561 7461 6e20 6e6f 726d 616c 206c 6577  uatan normal lew
000000b0: 6174 0a32 3032 362d 3130 2d31 3620 3038  at.2026-10-16 08
```

Sesuai deskripsi, ambil tiga digit milidetik hanya dari baris MERPATI. Baca angkanya sebagai bilangan desimal, lalu konversikan menjadi karakter ASCII.

| Milidetik | Nilai desimal | Karakter |
|---|---|---|
| 073 | 73 | I |
| 084 | 84 | T |
| 070 | 70 | F |
| 101 | 101 | e |
| 115 | 115 | s |

| | | |
|---|---|---|
| 116 | 116 | t |

Karakter awal membentuk ITFest, sehingga pendekatan ini sesuai dengan format flag. Gabungkan seluruh karakter sesuai urutan kemunculannya dalam file.

```
from pathlib import Path
import re

data = Path("forensic_4_jadwal_kereta.log").read_text()
milliseconds = re.findall(

   r"\d{2}:\d{2}:\d{2}\.(\d{3})

                                                    MERPATI


",
   data,
)
flag = "".join(chr(int(value, 10)) for value in milliseconds)
print(flag)
```

Regex menangkap angka milidetik tepat sebelum label [MERPATI]. Konversi int(value, 10) memastikan angka dengan nol di depan tetap dibaca sebagai desimal.

```
~/Documents/CTF/DIGFOR/Topeng Kayu dari Seberang
❯ python3 solve.py
ITFest2026{w4ktu_k3d4t4ng4n_k3r3t4_y4ng_b3rb1s1k_44d5}
```

**ITFest2026{w4ktu_k3d4t4ng4n_k3r3t4_y4ng_b3rb1s1k_44d5}**

# Reverse Engineering

## Saksi Bisu Lorong Gelap

| | |
|---|---|
| **Deskripsi** | Operator telepon di pos perbatasan sengaja menyilang delapan kabel tembaga di dalam kotak sambungan. Kabel pertama ditukar ke posisi kelima, kabel kedua ke posisi ketujuh, dan seterusnya. Pelajari skema persilangan kabel di dalam berkas cetak biru ini untuk mengembalikan suara asli yang teracak. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/1126225a5f02a7acb520e30a8892657f/re_4_kotak_kabel.py |
| **Flag** | ITFest2026{k4b3l_t3l3p0n_y4ng_d1s1l4ng_p4n1t14_40d4} |

Berkas Python menyediakan daftar byte SUARA_TERACAK, pemetaan PETA_KABEL, dan fungsi silang_kabel(). Delapan kabel merepresentasikan delapan bit dalam satu byte.

```
PETA_KABEL = [4, 6, 0, 2, 7, 1, 5, 3]

def silang_kabel(byte_masuk):
    keluar = 0
    for pos_baru, pos_lama in enumerate(PETA_KABEL):
        bit = (byte_masuk >> pos_lama) & 1
        keluar |= (bit << pos_baru)
    return keluar
```

Posisi bit dihitung dari kanan, mulai dari nol. Fungsi mengambil bit input di posisi PETA_KABEL[i] dan meletakkannya di posisi output i.

| Posisi bit input | Posisi bit output |
|---|---|
| 4 | 0 |
| 6 | 1 |
| 0 | 2 |
| 2 | 3 |
| 7 | 4 |
| 1 | 5 |
| 5 | 6 |
| 3 | 7 |

Seluruh posisi 0–7 muncul tepat sekali, sehingga transformasi ini adalah permutasi bit yang dapat dibalik tanpa kehilangan informasi. Untuk memulihkan byte asli, ambil bit output di posisi i, lalu kembalikan ke posisi PETA_KABEL[i]:

```
original |= ((value >> i) & 1) << PETA_KABEL[i]
```

Sebagai contoh, byte pertama data teracak adalah 134 (10000110 dalam biner). Setelah posisi bit dikembalikan, hasilnya 73 (01001001), yaitu karakter ASCII I.

```
from pathlib import Path
import runpy

challenge = runpy.run_path(str(Path(__file__).with_name("re_4_kotak_kabel.py")))

def restore(value):
    original = 0
    for new_position, old_position in enumerate(challenge["PETA_KABEL"]):
        original |= ((value >> new_position) & 1) << old_position
    return original

assert all(challenge["silang_kabel"](restore(value)) == value for value in range(256))
print(bytes(map(restore, challenge["SUARA_TERACAK"])).decode())
```

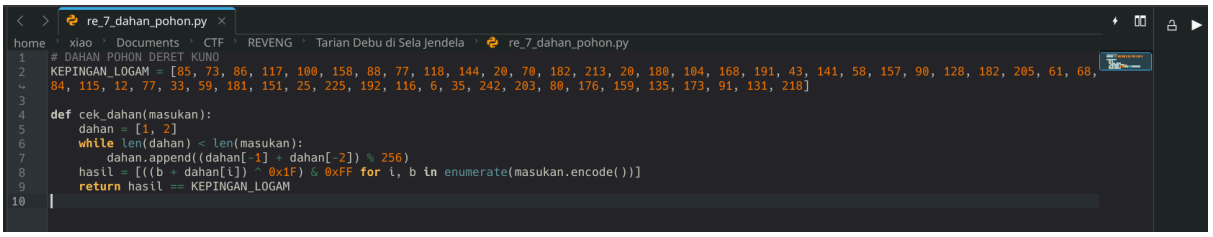
```
~/Documents/CTF/REVENG/Saksi Bisu Lorong Gelap
❯ python3 solve.py
ITFest2026{k4b3l_t3l3p0n_y4ng_d1s1l4ng_p4n1t14_40d4}
```

**ITFest2026{k4b3l_t3l3p0n_y4ng_d1s1l4ng_p4n1t14_40d4}**

# Tarian Debu di Sela Jendela

| | |
|---|---|
| **Deskripsi** | Dahan pohon tua itu selalu tumbuh bercabang mengikuti jumlah dua dahan di bawahnya. Sang penjaga hutan menyembunyikan kepingan logam di setiap cabangnya setelah disiram dengan air garam yang takarannya mengikuti jumlah dahan pada ketinggian tersebut. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/9af83d9597cf946919fda3a3ec25eb31/re_7_dahan_pohon.py |
| **Flag** | ITFest2026{p0h0n_b3rc4b4ng_m3ng1kut1_d3r3t_kuno_70a7} |

Berkas Python menyediakan daftar byte target KEPINGAN_LOGAM dan fungsi cek_dahan(masukan). Tujuannya adalah memulihkan string yang menghasilkan daftar target tersebut. Analisis berikut didasarkan pada kode sumber yang tersedia.




Fungsi pemeriksa membentuk deret yang dimulai dari 1, 2. Nilai berikutnya adalah jumlah dua nilai sebelumnya modulo 256:

```
dahan = [1, 2]
while len(dahan) < len(masukan):
   dahan.append((dahan[-1] + dahan[-2]) % 256)
```

Awal deretnya adalah:

```
1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 121, ...
```

Setiap byte masukan kemudian ditambah nilai deret pada posisi yang sama, di-XOR dengan 0x1F, dan dibatasi menjadi delapan bit:

```
hasil = [((b + dahan[i]) ^ 0x1F) & 0xFF for i, b in enumerate(masukan.encode())]
return hasil == KEPINGAN_LOGAM
```

Misalkan p[i] adalah byte asli, d[i] nilai deret, dan c[i] byte target. Transformasinya adalah:

```
c[i] = ((p[i] + d[i]) XOR 0x1F) AND 0xFF
```

XOR dapat dibalik menggunakan kunci yang sama. Penjumlahan dibalik dengan pengurangan modulo 256, sehingga:

```
p[i] = ((c[i] XOR 0x1F) - d[i]) mod 256
```

Sebagai contoh, byte target pertama adalah 85 atau 0x55. Nilai 0x55 XOR 0x1F adalah 74, lalu dikurangi nilai deret pertama (1) menghasilkan 73, yaitu karakter ASCII I.

Daftar target memiliki 53 elemen. Karena flag hasil pemulihan berupa ASCII, setiap karakter direpresentasikan oleh satu byte dan solver membentuk 53 nilai deret.

```
from re_7_dahan_pohon import KEPINGAN_LOGAM, cek_dahan

branches = [1, 2]
while len(branches) < len(KEPINGAN_LOGAM):
   branches.append((branches[-1] + branches[-2]) % 256)

flag = bytes(
   ((value ^ 0x1F) - branch) % 256
   for value, branch in zip(KEPINGAN_LOGAM, branches)
).decode()

assert cek_dahan(flag)
print(flag)
```

```
~/Documents/CTF/REVENG/Tarian Debu di Sela Jendela
❯ python3 solve.py
ITFest2026{p0h0n_b3rc4b4ng_m3ng1kut1_d3r3t_kuno_70a7}
```

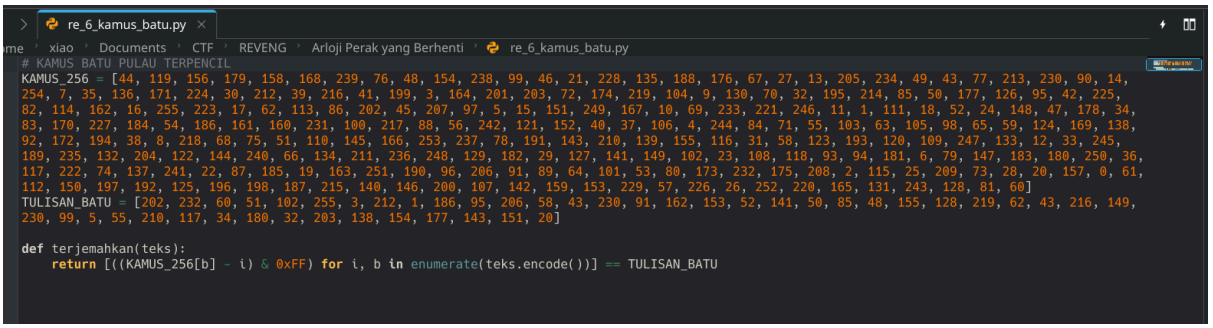
Pengecekan assert cek_dahan(flag) berhasil, sehingga string hasil pemulihan menghasilkan seluruh byte target saat dimasukkan kembali ke fungsi pemeriksa asli.

**ITFest2026{p0h0n_b3rc4b4ng_m3ng1kut1_d3r3t_kuno_70a7}**

# Arloji Perak yang Berhenti

| | |
|---|---|
| **Deskripsi** | Penduduk pulau terpencil itu menukar seluruh huruf di abjad mereka menggunakan sebuah tabel kamus batu berukuran 256 kotak. Setiap huruf yang masuk akan digantikan oleh simbol yang berada di kotak rujukan, lalu dikurangi nomor urut kedatangannya. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/a25f8c0c3d99bfcb7b3723a7e441d8a7/re_6_kamus_batu.py |
| **Flag** | ITFest2026{k4mus_b4h4s4_pul4u_t3rlup4k4n_60f6} |

Baca fungsi terjemahkan() terlebih dahulu.




Untuk karakter pada indeks i, fungsi mengambil byte input b, memakai KAMUS_256[b], menguranginya dengan i, lalu membandingkan hasilnya dengan TULISAN_BATU[i]:

```
target[i] = (KAMUS_256[b] - i) mod 256
```

Kita tidak perlu menebak flag. Rumus tersebut dapat dibalik menjadi KAMUS_256[b] = (target[i] + i) mod 256. Karena kamus berisi setiap nilai byte tepat satu kali, pencarian indeks menghasilkan satu byte asli untuk setiap target.

Byte pertama memberi karakter I, dan seluruh hasil terbaca sebagai flag. Pengecekan ulang dengan fungsi asli memastikan inversnya benar.

Setelah rumus dibalik, setiap byte hasil dikonversi menjadi karakter ASCII. Solver berikut mengotomatisasi perhitungan dan menjalankan pemeriksaan asli sebagai validasi:

```
import runpy

m=runpy.run_path('re_6_kamus_batu.py'); f=bytes(m['KAMUS_256'].index((c+i)&255) for i,c in enumerate(m['TULISAN_BATU'])).decode(); assert m['terjemahkan'](f); print(f)
```

Hasil script tidak hanya dicetak, tetapi juga diverifikasi dengan fungsi pemeriksa pada berkas challenge. Dengan begitu, string yang didapat benar-benar memenuhi kondisi validasi, bukan sekadar teks yang kebetulan terbaca.

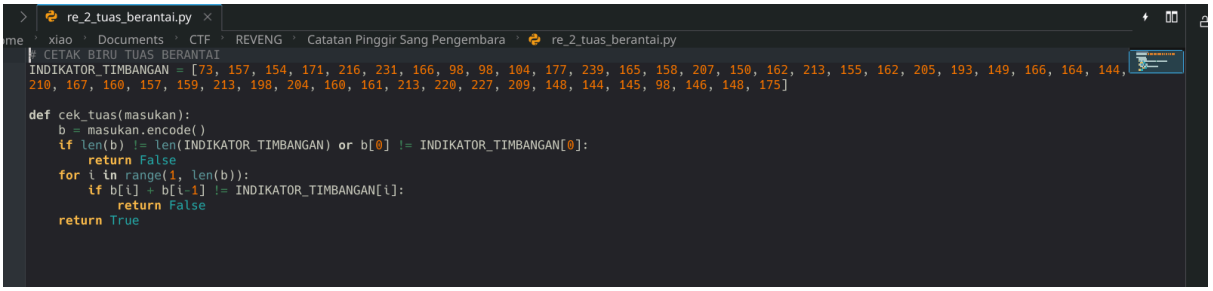
```
~/Documents/CTF/REVENG/Arloji Perak yang Berhenti
❯ python3 solve.py
ITFest2026{k4mus_b4h4s4_pul4u_t3rlup4k4n_60f6}
```

**ITFest2026{k4mus_b4h4s4_pul4u_t3rlup4k4n_60f6}**

# Catatan Pinggir Sang Pengembara

| | |
|---|---|
| **Deskripsi** | Gerbang besi ini memiliki puluhan tuas yang saling terhubung satu sama lain. Menarik tuas pertama akan ikut menggerakkan tuas di sebelahnya. Kamu diberikan cetak biru mekanisme roda giginya; hitunglah posisi awal setiap tuas agar seluruh indikator timbangan tepat berhenti di angka yang tertera. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/a25f8c0c3d99bfcb7b3723a7e441d8a7/re_6_kamus_batu.py |
| **Flag** | ITFest2026{t1mb4ng4n_b3s1_s4l1ng_m3ngunc1_20b2} |

Fungsi cek_tuas() memperlihatkan bahwa indikator pertama sama dengan byte pertama input. Setiap indikator berikutnya adalah jumlah byte berurutan:




```
indikator[0] = b[0]
indikator[i] = b[i] + b[i-1]
```

Maka byte pertama diambil langsung, lalu setiap byte berikutnya dihitung dengan pengurangan b[i] = indikator[i] - b[i-1].

Contohnya indikator awal 73, 157, 154 menghasilkan byte 73, 84, 70, yaitu ITF. Hasil lengkap dikonversi dari byte ke ASCII dan dimasukkan kembali ke cek_tuas().

Setelah rumus dibalik, setiap byte hasil dikonversi menjadi karakter ASCII. Solver berikut mengotomatisasi perhitungan dan menjalankan pemeriksaan asli sebagai validasi:

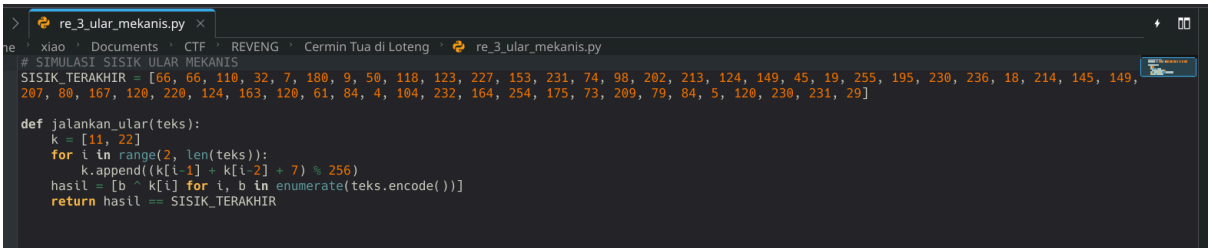
```
import runpy
m=runpy.run_path('re_2_tuas_berantai.py'); t=m['INDIKATOR_TIMBANGAN']; b=[t[0]]
for x in t[1:]: b.append(x-b[-1])
f=bytes(b).decode(); assert m['cek_tuas'](f); print(f)
```

```
~/Documents/CTF/REVENG/Catatan Pinggir Sang Pengembara
❯ python3 solve.py
ITFest2026{t1mb4ng4n_b3s1_s4l1ng_m3ngunc1_20b2}
```

**ITFest2026{t1mb4ng4n_b3s1_s4l1ng_m3ngunc1_20b2}**

# Cermin Tua di Loteng

| | |
|---|---|
| **Deskripsi** | Seekor ular mekanis merayap di atas papan catur. Setiap kali ia berpindah satu kotak, warna sisik di tubuhnya berubah mengikuti hasil penjumlahan dua kotak sebelumnya. Jangan hanya membaca warna sisiknya saat ia tertidur diam, tetapi jalankan pergerakannya hingga langkah terakhir. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/b8e0dbac8ef2409277d0ec9b6c41894d/re_3_ular_mekanis.py |
| **Flag** | ITFest2026{ul4r_y4ng_b3rg4nt1_kul1t_s3t14p_d3t1k_30c3} |




Pemeriksa membangun deret kunci dari [11, 22] dengan rumus k[i] = (k[i-1] + k[i-2] + 7) mod 256.

Setelah itu setiap byte input di-XOR dengan kunci pada posisi yang sama dan dibandingkan dengan SISIK_TERAKHIR.

XOR adalah operasi involutif: x XOR k XOR k = x. Jadi setelah membangun deret kunci sepanjang data, byte asli diperoleh dari SISIK_TERAKHIR[i] XOR k[i].

Dua byte pertama, misalnya, menghasilkan 66 XOR 11 = 73 (I) dan 66 XOR 22 = 84 (T). String hasil kemudian diverifikasi oleh jalankan_ular().

```
import runpy
m=runpy.run_path('re_3_ular_mekanis.py'); k=[11,22]
while len(k)<len(m['SISIK_TERAKHIR']): k.append((k[-1]+k[-2]+7)%256)

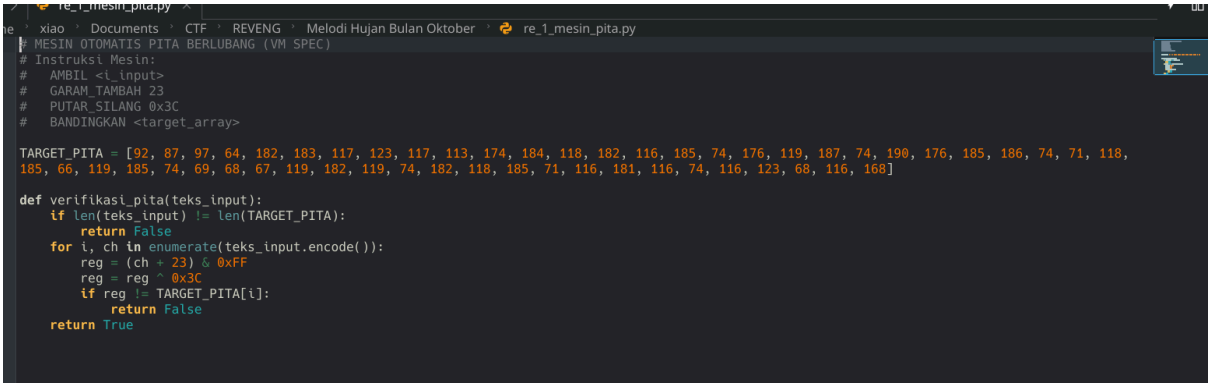
f=bytes(x^y for x,y in zip(m['SISIK_TERAKHIR'],k)).decode(); assert m['jalankan_ular'](f); print(f)
```

```
~/Documents/CTF/REVENG/Cermin Tua di Loteng
❯ python3 solve.py
ITFest2026{ul4r_y4ng_b3rg4nt1_kul1t_s3t14p_d3t1k_30c3}
```

**ITFest2026{ul4r_y4ng_b3rg4nt1_kul1t_s3t14p_d3t1k_30c3}**

# Melodi Hujan Bulan Oktober

| | |
|---|---|
| **Deskripsi** | Di ruang bawah tanah ditemukan sebuah mesin otomatis tua yang tidak mengenali bahasa manusia maupun bahasa mesin modern. Mesin ini hanya bekerja berdasarkan gulungan pita berlubang dengan empat perintah sederhana: ambil nilai, tambahkan garam, putar balik, dan bandingkan dengan deretan angka target. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/abd9e1049bf9e1b7c3f4660eb6087e8e/re_1_mesin_pita.py |
| **Flag** | ITFest2026{m3s1n_u4p_kuno_d3ng4n_bah4s4_s3nd1r1_10a1} |



```
xiao › Documents › CTF › REVENG › Melodi Hujan Bulan Oktober › re_1_mesin_pita.py
# MESIN OTOMATIS PITA BERLUBANG (VM SPEC)
# Instruksi Mesin:
#   AMBIL <i_input>
#   GARAM_TAMBAH 23
#   PUTAR_SILANG 0x3C
#   BANDINGKAN <target_array>

TARGET_PITA = [92, 87, 97, 64, 182, 183, 117, 123, 117, 113, 174, 184, 118, 182, 116, 185, 74, 176, 119, 187, 74, 190, 176, 185, 186, 74, 71, 118,
185, 66, 119, 185, 74, 69, 68, 67, 119, 182, 119, 74, 182, 118, 185, 71, 116, 181, 116, 74, 116, 123, 68, 116, 168]

def verifikasi_pita(teks_input):
    if len(teks_input) != len(TARGET_PITA):
        return False
    for i, ch in enumerate(teks_input.encode()):
        reg = (ch + 23) & 0xFF
        reg = reg ^ 0x3C
        if reg != TARGET_PITA[i]:
            return False
    return True
```

Komentar VM menjelaskan tiga operasi pada setiap karakter: ambil byte, tambah garam 23, lalu XOR dengan 0x3C. Fungsi verifikasi_pita() menuliskan transformasinya secara langsung:

```
target[i] = ((input[i] + 23) XOR 0x3C) mod 256
```

Operasi dibalik dari tahap terakhir ke tahap pertama. XOR dibalik dengan kunci yang sama, kemudian 23 dikurangkan modulo 256.

Pada target pertama, 92 XOR 0x3C = 96, lalu 96 - 23 = 73, yaitu I. Menerapkan rumus ke semua target menghasilkan flag.

Setelah rumus dibalik, setiap byte hasil dikonversi menjadi karakter ASCII. Solver berikut otomatis menghitung dan menjalankan pemeriksaan asli sebagai validasi:

```
import runpy

m=runpy.run_path('re_1_mesin_pita.py'); f=bytes(((x^0x3c)-23)%256 for x in
m['TARGET_PITA']).decode(); assert m['verifikasi_pita'](f); print(f)
```

```
~/Documents/CTF/REVENG/Melodi Hujan Bulan Oktober
❯ python3 solve.py
ITFest2026{m3s1n_u4p_kuno_d3ng4n_bah4s4_s3nd1r1_10a1}
```

**ITFest2026{m3s1n_u4p_kuno_d3ng4n_bah4s4_s3nd1r1_10a1}**

# Patung Batu di Dasar Danau

| | |
|---|---|
| **Deskripsi** | Selembar kertas dibagi menjadi sisi kiri dan sisi kanan. Sebanyak empat kali berturut-turut, sisi kanan kertas dicap dengan stempel garam lalu ditimpakan ke sisi kiri, kemudian posisi kiri dan kanan ditukar silang. Bukalah kembali lipatan tersebut dari putaran keempat mundur ke putaran pertama. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/5a569c6a832231aed3207ca93dde8d42/re_10_lipatan_kertas.py |
| **Flag** | ITFest2026{l1p4t4n_k1r1_d4n_k4n4n_s4l1ng_b3rtuk4r_00d0} |




lipat_kertas() membagi input menjadi L dan R, lalu menjalankan empat putaran:

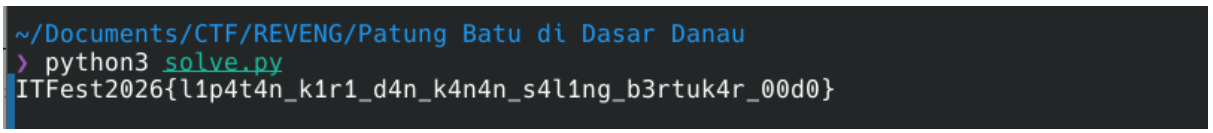
```
F(R) = (R + putaran*17) XOR 0xA5
L_baru = R
R_baru = L XOR F(R)
```

Ini adalah jaringan Feistel, sehingga dapat dibuka tanpa membalik fungsi stempel. Pada putaran terbalik, oldR = L_baru dan oldL = R_baru XOR F(L_baru).

Ciphertext hex dibagi menjadi dua bagian, lalu putaran 4, 3, 2, dan 1 dibatalkan. Hasilnya memiliki satu spasi padding karena panjang flag ganjil; spasi tersebut dihapus sebelum decode ASCII.

Setelah rumus direverse, setiap byte hasil dikonversi menjadi karakter ASCII. Solver berikut untuk mendapatkan flagnya:

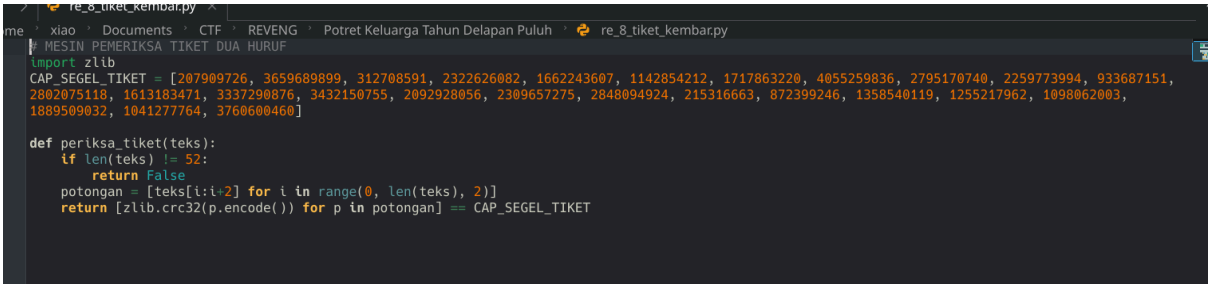
```
import runpy
m=runpy.run_path('re_10_lipatan_kertas.py'); q=bytes.fromhex(m['HASIL_LIPATAN_HEX']);
h=len(q)//2; L,R=q[:h],q[h:]
for r in range(4,0,-1): oldL=bytes(R[i]^m['fungsi_stempel'](L,r)[i] for i in range(h)); L,R=oldL,L
f=(L+R).rstrip(b' ').decode(); assert m['lipat_kertas'](f); print(f)
```




**ITFest2026{l1p4t4n_k1r1_d4n_k4n4n_s4l1ng_b3rtuk4r_00d0}**

# Potret Keluarga Tahun Delapan Puluh

| | |
|---|---|
| **Deskripsi** | Mesin pemeriksa tiket itu tidak memeriksa tiket secara utuh, melainkan memotongnya menjadi potongan-potongan kecil yang masing-masing hanya terdiri dari dua huruf berdampingan, lalu mencetak cap segel empat byte untuk setiap pasang hurufnya. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/cf49c522fab7aa390b849315ca2dd55f/re_8_tiket_kembar.py |
| **Flag** | ITFest2026{s3g3l_k3mb4r_dua_huruf_b3rp4s4ng4n_80b8} |




Pemeriksa memotong teks menjadi pasangan dua karakter dan menghitung zlib.crc32() untuk setiap pasangan.

Nilai CRC32 tidak praktis dibalik secara aljabar, tetapi ruang kandidatnya kecil: karakter flag berada pada ASCII printable (32 sampai 126), sehingga hanya ada $95^2 = 9025$ pasangan.

Buat tabel CRC32 -> pasangan untuk semua kandidat tersebut. Setiap nilai dalam CAP_SEGEL_TIKET kemudian dicari di tabel.

Pasangan pertama menghasilkan IT, berikutnya Fe, lalu st, sehingga hasil awal cocok dengan format flag. Setelah 26 pasangan digabungkan dan dimasukkan kembali ke periksa_tiket(), seluruh cap cocok.

Setiap byte hasil dari rumus yang telah dibalik akan diubah menjadi karakter ASCII. Solver ini berfungsi untuk mengotomatisasi perhitungan sekaligus memvalidasi pemeriksaan aslinya.

```
import runpy

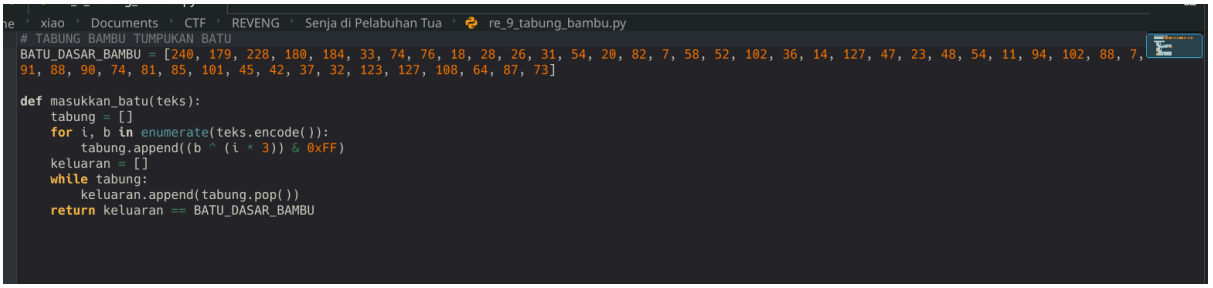
m=runpy.run_path('re_8_tiket_kembar.py'); z=__import__('zlib');
d={z.crc32(bytes((a,b))):bytes((a,b)) for a in range(32,127) for b in range(32,127)}; f=b''.join(d[c]
for c in m['CAP_SEGEL_TIKET']).decode(); assert m['periksa_tiket'](f); print(f)
```

```
~/Documents/CTF/REVENG/Potret Keluarga Tahun Delapan Puluh
❯ python3 solve.py
ITFest2026{s3g3l_k3mb4r_dua_huruf_b3rp4s4ng4n_80b8}
```

**ITFest2026{s3g3l_k3mb4r_dua_huruf_b3rp4s4ng4n_80b8}**

# Senja di Pelabuhan Tua

| | |
|---|---|
| **Deskripsi** | Anak-anak desa bermain menyusun batu ke dalam sebuah tabung bambu sempit yang hanya bisa dimasuki dan dikeluarkan dari lubang atasnya saja. Ikuti urutan instruksi keluar-masuk batu di dalam catatan ini untuk mengetahui batu mana saja yang tersisa di dasar bambu. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/8b3c88d61c7b699bc031683f6a400c59/re_9_tabung_bambu.py |
| **Flag** | ITFest2026{tumpuk4n_b4tu_d1_p1ngg1r_sung41_90c9} |




Fungsi masukkan_batu() melakukan dua langkah: byte input pada indeks i di-XOR dengan i*3, kemudian seluruh list dibalik menggunakan pop().

Karena target adalah hasil akhir stack, langkah pertama pemulihan adalah membalik urutan BATU_DASAR_BAMBU.

Setelah urutannya kembali, XOR dengan i*3 dibatalkan menggunakan operasi XOR yang sama.

Byte pertama setelah pembalikan dan XOR menghasilkan 73 (I), lalu byte berikutnya 84 (T). Semua byte hasil membentuk flag dan validasi fungsi asli berhasil.

Konversi ke karakter ASCII dilakukan pada setiap byte hasil setelah rumus dibalik. Guna mengotomatisasi kalkulasi dan memverifikasi pemeriksaan asli, digunakan solver di bawah ini.

```
import runpy

m=runpy.run_path('re_9_tabung_bambu.py'); a=list(reversed(m['BATU_DASAR_BAMBU']));
f=bytes(v^(i*3) for i,v in enumerate(a)).decode(); assert m['masukkan_batu'](f); print(f)
```

```
~/Documents/CTF/REVENG/Senja di Pelabuhan Tua
❯ python3 solve.py
ITFest2026{tumpuk4n_b4tu_d1_p1ngg1r_sung41_90c9}
```

**ITFest2026{tumpuk4n_b4tu_d1_p1ngg1r_sung41_90c9}**

# Sepasang Sepatu Usang

| | |
|---|---|
| **Deskripsi** | Arsitek gila itu membangun istana dengan seratus lorong yang tampak sangat meyakinkan, lengkap dengan peti harta karun tiruan di setiap ujung lorong untuk menjebak para pencuri tergesa-gesa. Hanya lorong yang memenuhi syarat keseimbangan angka kuno yang benar-benar terhubung ke ruang singgasana. |
| **Attachment** | https://ctf.it-festival2026.cloud/files/ca2d0ebf68fc2e4131d81843d45daa29/re_5_seratus_lorong.py |
| **Flag** | |

```
# LABIRIN SERATUS LORONG ISTANA

def lorong_1(kunci):
    # Lorong ilusi penjebak mesin
    target = [139, 102, 81, 91, 134, 122, 149, 99, 101, 84, 74, 89, 73, 92, 81, 126, 126, 85,
136, 112, 73, 53, 129, 136, 104, 83, 123, 126, 127, 92]
    return [b + 1 for b in kunci.encode()] == target

def lorong_2(kunci):
    # Lorong ilusi penjebak mesin
    target = [138, 123, 127, 143, 57, 50, 136, 61, 83, 69, 92, 119, 96, 82, 57, 113, 101, 132,
110, 109, 146, 150, 145, 108, 115, 85, 94, 122, 113, 66]
    return [b + 2 for b in kunci.encode()] == target

def lorong_3(kunci):
    # Lorong ilusi penjebak mesin
    target = [86, 79, 108, 87, 137, 105, 84, 50, 52, 119, 101, 133, 58, 136, 63, 103, 142, 146,
123, 110, 122, 124, 132, 139, 72, 144, 59, 110, 77, 127]
    return [b + 3 for b in kunci.encode()] == target

def lorong_4(kunci):
    # Lorong ilusi penjebak mesin
    target = [54, 95, 86, 147, 82, 144, 59, 99, 144, 116, 128, 62, 53, 70, 87, 61, 57, 82, 107,
129, 97, 83, 142, 52, 83, 91, 72, 83, 136, 91]
    return [b + 4 for b in kunci.encode()] == target

def lorong_5(kunci):
    # Lorong ilusi penjebak mesin
    target = [69, 62, 73, 66, 64, 146, 54, 117, 115, 148, 106, 127, 139, 95, 141, 87, 144, 104,
83, 71, 102, 79, 104, 99, 134, 85, 72, 114, 74, 77]
    return [b + 5 for b in kunci.encode()] == target

def lorong_6(kunci):
    # Lorong ilusi penjebak mesin
    target = [143, 126, 143, 106, 57, 52, 70, 140, 67, 60, 91, 140, 59, 56, 93, 88, 136, 121, 97,
114, 68, 72, 123, 124, 109, 91, 91, 105, 85, 72]
    return [b + 6 for b in kunci.encode()] == target

def lorong_7(kunci):
```

```
    # Lorong ilusi penjebak mesin
    target = [137, 133, 106, 87, 149, 57, 67, 57, 140, 111, 149, 136, 93, 76, 107, 111, 82, 122,
90, 97, 111, 50, 84, 141, 50, 111, 140, 149, 99, 97]
    return [b + 7 for b in kunci.encode()] == target

def lorong_8(kunci):
    # Lorong ilusi penjebak mesin
    target = [63, 114, 95, 92, 102, 60, 91, 142, 144, 134, 144, 108, 129, 94, 109, 109, 62, 92,
127, 150, 127, 98, 74, 87, 115, 109, 91, 114, 127, 116]
    return [b + 8 for b in kunci.encode()] == target

def lorong_9(kunci):
    # Lorong ilusi penjebak mesin
    target = [142, 74, 117, 86, 124, 121, 74, 80, 120, 85, 69, 99, 108, 118, 144, 103, 132, 84,
117, 129, 51, 104, 105, 115, 116, 68, 129, 55, 62, 128]
    return [b + 9 for b in kunci.encode()] == target

def lorong_10(kunci):
    # Lorong ilusi penjebak mesin
    target = [78, 67, 134, 58, 80, 117, 94, 62, 58, 147, 129, 73, 119, 137, 75, 111, 105, 64, 83,
150, 109, 82, 112, 103, 92, 79, 142, 100, 75, 66]
    return [b + 10 for b in kunci.encode()] == target

def lorong_11(kunci):
    # Lorong ilusi penjebak mesin
    target = [55, 130, 50, 127, 134, 79, 137, 137, 89, 138, 145, 105, 124, 94, 50, 89, 108, 99,
68, 139, 131, 112, 117, 97, 149, 120, 109, 59, 124, 50]
    return [b + 11 for b in kunci.encode()] == target

def lorong_12(kunci):
    # Lorong ilusi penjebak mesin
    target = [115, 62, 118, 74, 119, 84, 121, 138, 132, 83, 129, 131, 92, 70, 130, 139, 147, 97,
59, 111, 131, 70, 51, 94, 83, 93, 143, 54, 115, 148]
    return [b + 12 for b in kunci.encode()] == target

def lorong_13(kunci):
    # Lorong ilusi penjebak mesin
    target = [116, 74, 93, 109, 139, 81, 77, 112, 150, 69, 137, 50, 123, 79, 122, 60, 132, 61,
108, 95, 119, 94, 79, 59, 143, 102, 133, 137, 82, 105]
    return [b + 13 for b in kunci.encode()] == target

def lorong_14(kunci):
    # Lorong ilusi penjebak mesin
    target = [123, 81, 148, 80, 105, 124, 103, 147, 115, 50, 144, 147, 111, 140, 63, 131, 87,
108, 68, 100, 118, 78, 148, 71, 77, 68, 78, 108, 56, 115]
    return [b + 14 for b in kunci.encode()] == target

def lorong_15(kunci):
    # Lorong ilusi penjebak mesin
    target = [73, 120, 97, 103, 88, 121, 137, 145, 57, 88, 62, 58, 76, 121, 118, 123, 118, 99,
147, 102, 140, 103, 121, 116, 125, 90, 78, 130, 133, 147]
    return [b + 15 for b in kunci.encode()] == target

def lorong_16(kunci):
```

```
    # Lorong ilusi penjebak mesin
    target = [148, 86, 59, 90, 124, 131, 96, 91, 73, 98, 118, 126, 142, 146, 118, 143, 130, 58,
88, 111, 138, 138, 91, 56, 131, 149, 110, 58, 94, 88]
    return [b + 16 for b in kunci.encode()] == target

def lorong_17(kunci):
    # Syarat keseimbangan kuno: 17 * 3 == 51
    target = [20, 31, 13, 48, 58, 63, 121, 123, 121, 125, 66, 58, 122, 57, 124, 41, 64, 38, 59,
124, 53, 63, 64, 38, 59, 127, 55, 58, 64, 38, 58, 127, 63, 64, 38, 49, 127, 55, 127, 53, 38, 59,
64, 55, 127, 53, 46, 38, 128, 123, 48, 128, 72]
    return [((b ^ 0x42) + 9) for b in kunci.encode()] == target

def lorong_18(kunci):
    # Lorong ilusi penjebak mesin
    target = [125, 94, 130, 82, 69, 145, 123, 129, 135, 65, 114, 80, 128, 129, 52, 81, 130, 112,
129, 118, 138, 53, 65, 119, 125, 122, 99, 130, 90, 149]
    return [b + 18 for b in kunci.encode()] == target

def lorong_19(kunci):
    # Lorong ilusi penjebak mesin
    target = [95, 66, 53, 60, 87, 105, 114, 62, 104, 77, 134, 139, 131, 120, 96, 82, 136, 60,
133, 142, 59, 121, 136, 127, 121, 58, 134, 96, 52, 123]
    return [b + 19 for b in kunci.encode()] == target

def lorong_20(kunci):
    # Lorong ilusi penjebak mesin
    target = [145, 79, 67, 123, 125, 60, 104, 55, 128, 112, 69, 105, 114, 108, 123, 50, 96, 86,
71, 112, 84, 101, 72, 92, 91, 127, 86, 75, 126, 101]
    return [b + 20 for b in kunci.encode()] == target

def lorong_21(kunci):
    # Lorong ilusi penjebak mesin
    target = [122, 142, 82, 78, 74, 86, 120, 110, 102, 122, 77, 56, 63, 80, 61, 91, 105, 61, 132,
147, 76, 126, 143, 81, 60, 110, 122, 141, 59, 102]
    return [b + 21 for b in kunci.encode()] == target

def lorong_22(kunci):
    # Lorong ilusi penjebak mesin
    target = [98, 108, 55, 85, 80, 93, 119, 115, 103, 126, 87, 129, 89, 119, 91, 92, 93, 110,
121, 137, 117, 124, 109, 57, 63, 92, 93, 51, 102, 102]
    return [b + 22 for b in kunci.encode()] == target

def lorong_23(kunci):
    # Lorong ilusi penjebak mesin
    target = [83, 128, 150, 55, 53, 93, 105, 61, 117, 111, 105, 120, 82, 137, 58, 56, 76, 94, 68,
121, 83, 107, 85, 119, 150, 90, 73, 150, 120, 87]
    return [b + 23 for b in kunci.encode()] == target

def lorong_24(kunci):
    # Lorong ilusi penjebak mesin
    target = [140, 62, 71, 98, 121, 73, 100, 120, 118, 85, 101, 103, 111, 56, 116, 101, 57, 148,
110, 140, 135, 57, 61, 135, 150, 88, 64, 129, 118, 105]
    return [b + 24 for b in kunci.encode()] == target
```

```
def gerbang_utama(kunci, nomor_lorong):
  if nomor_lorong * 3 == 51:
    return lorong_17(kunci)
  return False
```

Ada banyak fungsi lorong_1 sampai lorong_24, tetapi gerbang_utama() hanya memanggil lorong jika nomor_lorong * 3 == 51.

Persamaan ini menentukan satu-satunya lorong valid, yaitu nomor 17. Lorong lain adalah decoy.

lorong_17() memeriksa transformasi target[i] = (input[i] XOR 0x42) + 9. Balik dengan mengurangi 9 terlebih dahulu, lalu XOR dengan 0x42.

Target pertama 20 menghasilkan (20-9) XOR 0x42 = 73, karakter I. Setelah seluruh byte dipulihkan, masukan diuji melalui gerbang_utama(flag, 17).

Setelah rumus dibalik, setiap byte dari hasil perhitungan dikonversi menjadi karakter ASCII.

Proses validasi kemudian diotomatisasi menggunakan solver berikut untuk menjalankan pemeriksaan asli.

```
import runpy

m=runpy.run_path('re_5_seratus_lorong.py');
t=[20,31,13,48,58,63,121,123,121,125,66,58,122,57,124,41,64,38,59,124,53,63,64,38,59,127,55,58
,64,38,58,127,63,64,38,49,127,55,127,53,38,59,64,55,127,53,46,38,128,123,48,128,72];
f=bytes((x-9)^0x42 for x in t).decode(); assert m['gerbang_utama'](f,17); print(f)
```

```
~/Documents/CTF/REVENG/Sepasang Sepatu Usang
❯ python3 solve.py
ITFest2026{s3r1bu_p1ntu_p4lsu_s4tu_j4l4n_pul4ng_50e5}
```

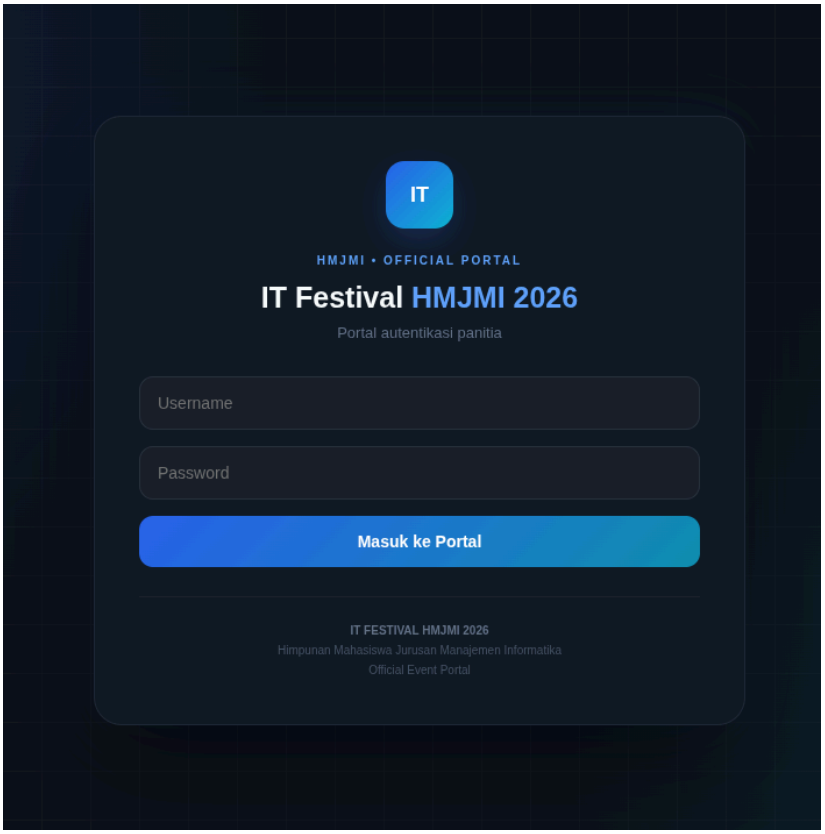
**ITFest2026{s3r1bu_p1ntu_p4lsu_s4tu_j4l4n_pul4ng_50e5}**

# Web Exploitation

## Gerbang Penjara Jiwa

| | |
|---|---|
| **Deskripsi** | Penjaga gerbang ini memasang segel mantra tua di pintu masuknya. Konon, pintu akan terbuka lebar bukan karena kekerasan, melainkan karena bisikan logika yang mengecoh keyakinan sang penjaga. Temukan celah pada lembar catatan masuk mereka. |
| **Attachment** | https://porrtall.vercel.app/ |
| **Flag** | **ITFEST26{sql1_byp4ss_auth}** |

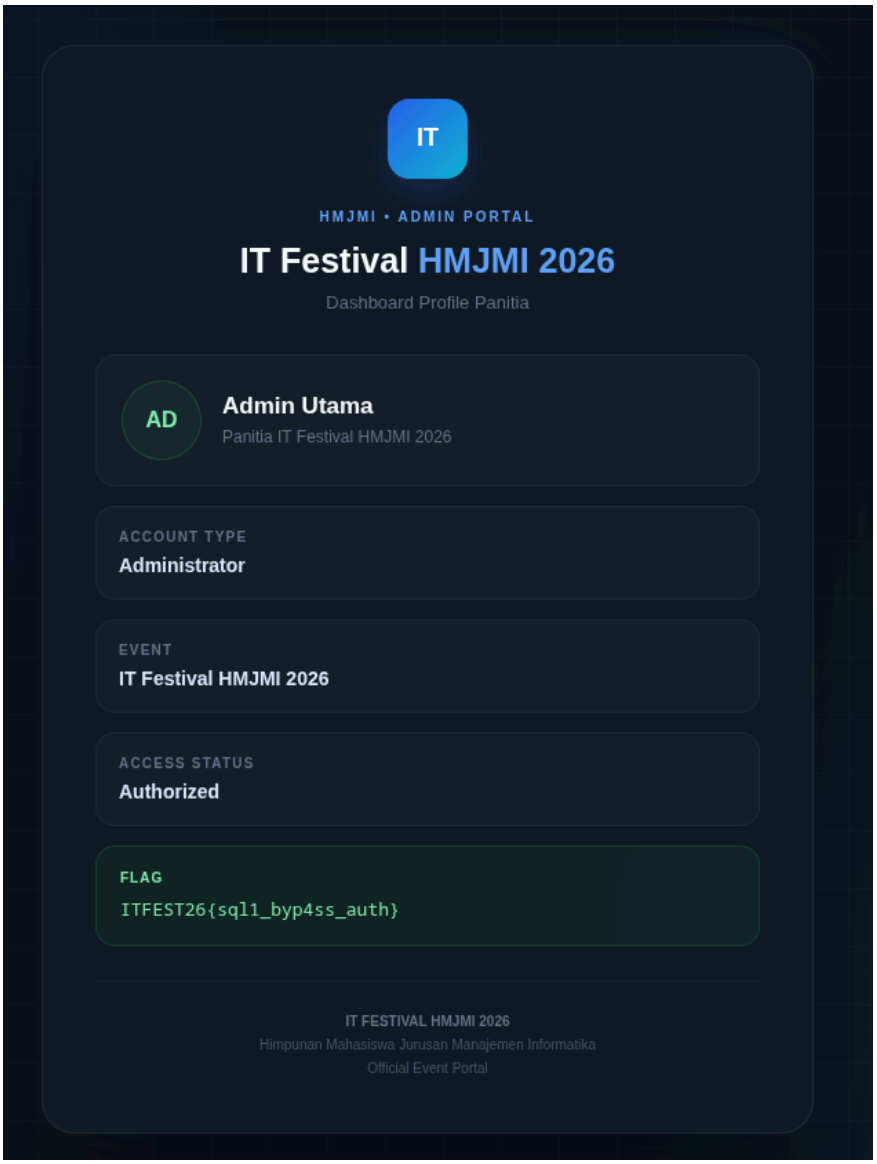
Saat masuk ke website terdapat sebuah form login. Hal pertama yang langsung terpikirkan saat melihat form login seperti ini adalah sql injection.




Jadi dengan memasukkan **admin' --** sebagai username, password masukkan random saja. Sesuai dugaan, celah keamanan pada website ini adalah penanganan sqli yang kurang tepat, membuat database tidak bisa membedakan mana data dan mana perintah.

Dengan menambahkan tanda kutip database akan menganggapnya sebagai penutup string sehingga strip dua setelahnya akan dianggap sebagai perintah. Karena strip 2

dalam mysql adalah perintah untuk komen, maka semua kode setelahnya akan dianggap komentar dan login akan diperbolehkan.

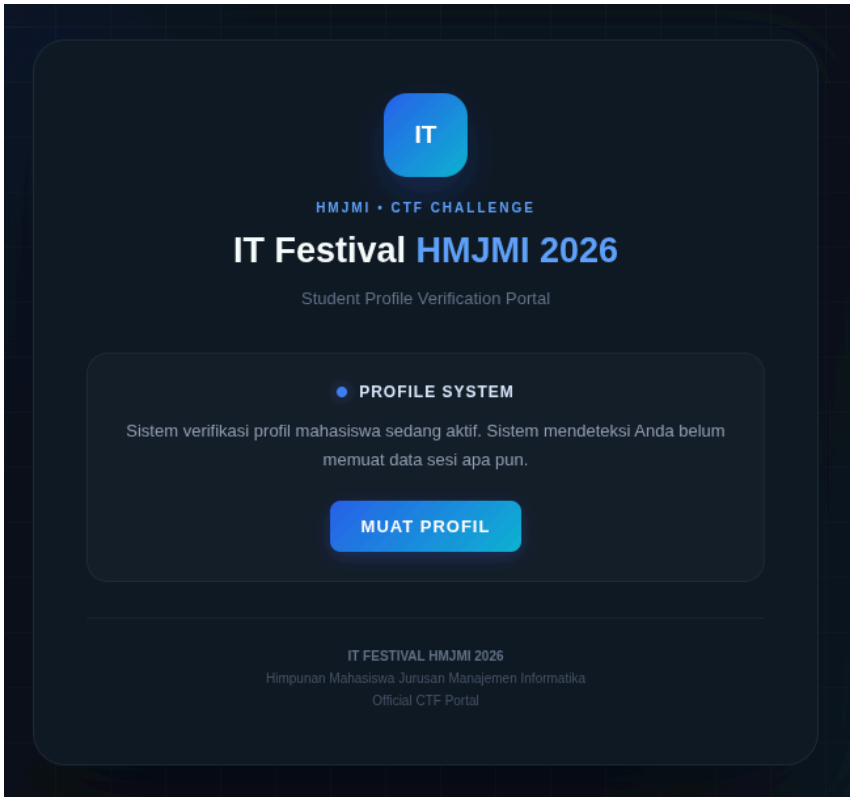



**ITFEST26{sql1_byp4ss_auth}**

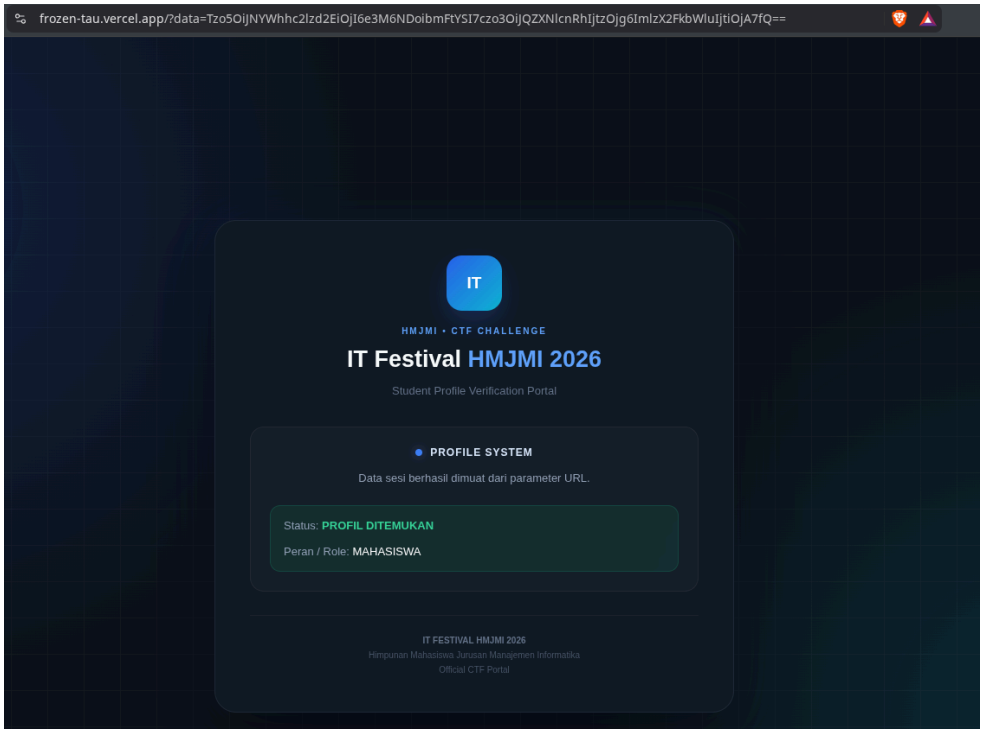
# Kotak Peti Keramat

| | |
|---|---|
| **Deskripsi** | Sebuah peti kiriman dari negeri seberang dibiarkan terbuka sambungannya. Ada bagian rantai pasokan yang sudah berkarat dan rapuh jika ditarik dari arah luar. Perhatikan daftar barang bawaan yang masuk ke pelabuhan. |
| **Attachment** | https://frozen-tau.vercel.app/ |
| **Flag** | **ITFEST26{d3s3r1al1z4t10n_pwn}** |

Saat website dibuka terdapat sebuah menu untuk muat profil.

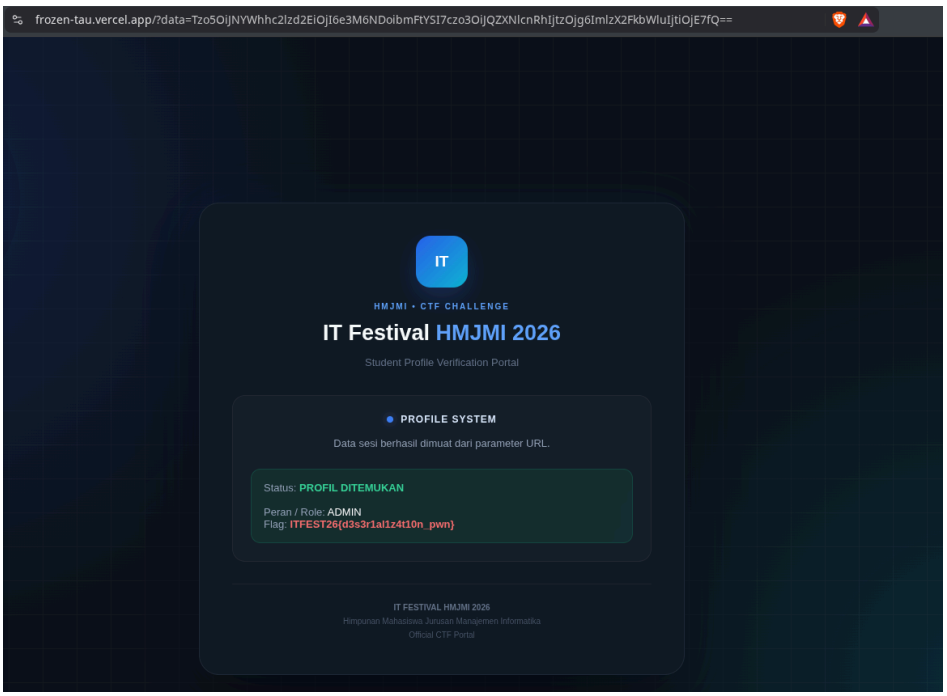



Saat muat profil diklik, akan terbuat profil mahasiswa dengan sebuah parameter ?data pada url.




Celahnya di sini adalah parameter data di url bisa dilihat dari client side, dengan kata lain celah keamanannya adalah **Insecure Deserialization**.

Base64 pada parameter data ketika di-decode hasilnya adalah **O:9:"Mahasiswa":2:{s:4:"nama";s:7:"Peserta";s:8:"is_admin";b:0;}**. Cuma perlu ubah nilai boolean is_admin jadi 1 lalu encode ulang ke bas64, masukkan ke parameter maka flag akan didapat.



**ITFEST26{d3s3r1al1z4t10n_pwn}**

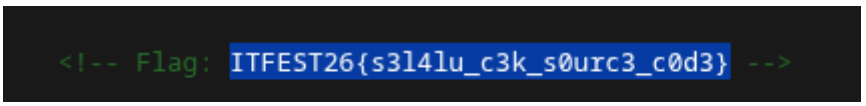
# Celah Tirai Panggung

| | |
|---|---|
| **Deskripsi** | Pementasan terlihat megah dari depan, namun tirai belakangnya sengaja dibiarkan sedikit tersingkap oleh sang dalang. Jangan percaya sepenuhnya pada apa yang tampak di layar utama; intip lembar skrip di balik tirai. |
| **Attachment** | https://develop-inspect.vercel.app/ |
| **Flag** | **ITFEST26{s3l4lu_c3k_s0urc3_c0d3}** |

Saat website dibuka hanya terdapat sebuah box selamat datang biasa tanpa tombol apapun yang bisa di-klik.




Pada challenge ini celahnya sangat simpel, flag-nya di-reveal di source code website.

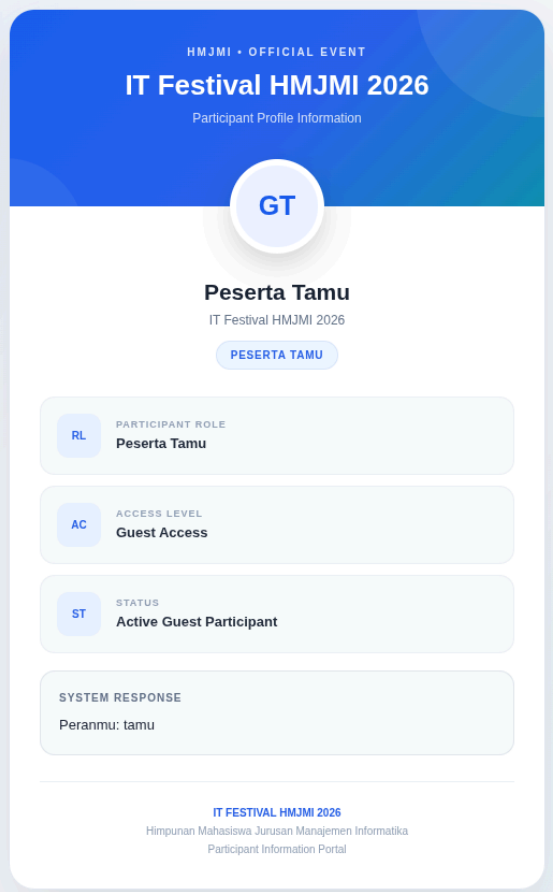



**ITFEST26{s3l4lu_c3k_s0urc3_c0d3}**

# Karcis Singgasana Maharaja

| | |
|---|---|
| **Deskripsi** | Kursi kehormatan di barisan depan hanya diberikan kepada tamu berkalung lencana khusus. Namun, lencana tersebut disimpan ceroboh di dalam saku mantel sementaramu. Coba tukar atributnya sebelum pintu aula dikunci. |
| **Attachment** | https://vip-changer.vercel.app/ |
| **Flag** | **ITFEST26{c00k13_m4n1pUlat10n_3asy}** |

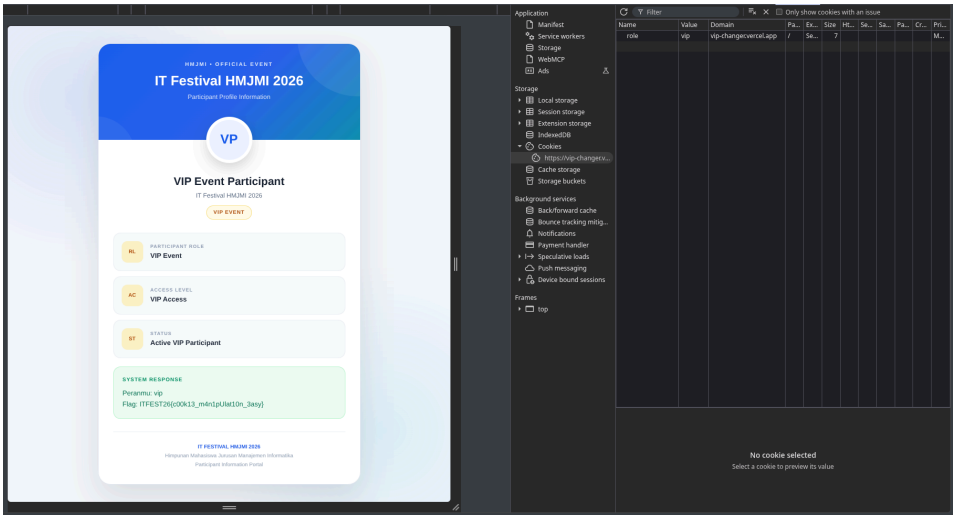
Saat website dibuka terdapat box yang mengandung informasi tentang partisipan IT Festival serta role-nya.




Karena di sini ada hal yang berkaitan dengan role, jadi langsung saja cek cookie dan ternyata ada sebuah cookie dengan value tamu.

| Name | Value | Domain | Pa... | Ex... | Size | Ht... | Se... | Sa... | Pa... | Cr... | Pri... |
|---|---|---|---|---|---|---|---|---|---|---|---|
| role | tamu | vip-changer.vercel.app | / | Se... | 8 | | | | | | M... |

Dari domain sudah ada hint “vip-changer...” maka coba ubah value dari tamu ke vip.




Setelah value diubah dari tamu ke vip, role ikut terdeteksi sebagai vip sehingga flag dapat terlihat.

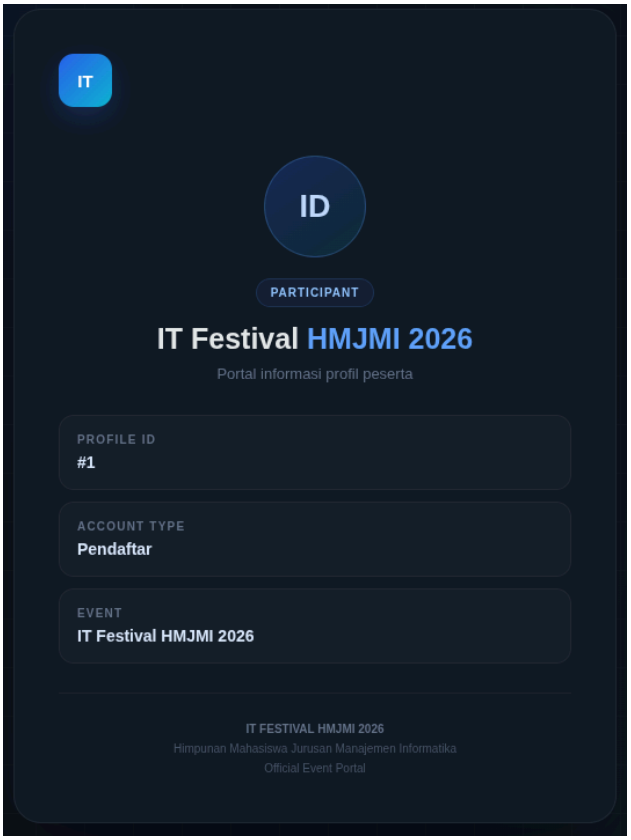
Jadi celah keamanan pada website kali ini adalah Broken Access Control. Tidak ada validasi otorisasi di sisi server, sehingga server mempercayai begitu saja nilai cookie role yang dikirim dari client, tanpa mencocokkannya ke data authorization (misal session atau database) di server.

**ITFEST26{c00k13_m4n1pUlat10n_3asy}**

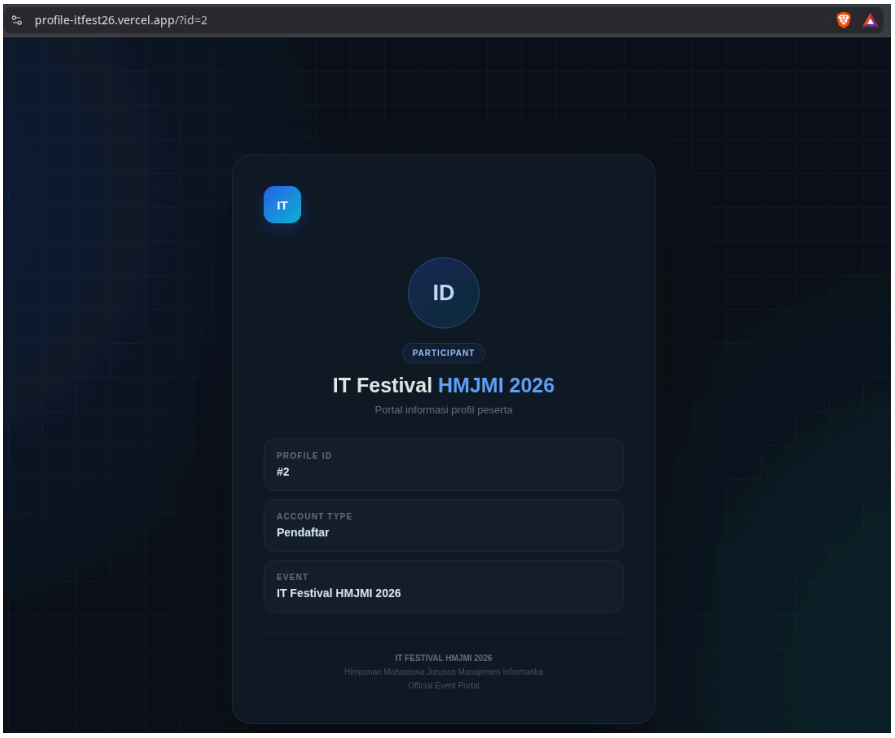
# Topeng Wajah Bayangan

| | |
|---|---|
| **Deskripsi** | Di balai topeng ini tahun 2026, siapa pun bisa menyamar menjadi orang lain sekadar dengan mengubah nomor registrasi lencana di tangan. Amati alur kunjungan para bangsawan lain dan ambil alih tempat mereka. |
| **Attachment** | https://profile-itfest26.vercel.app/ |
| **Flag** | **ITFEST26{1d0r_4cc3ss_gr4nt3d}** |

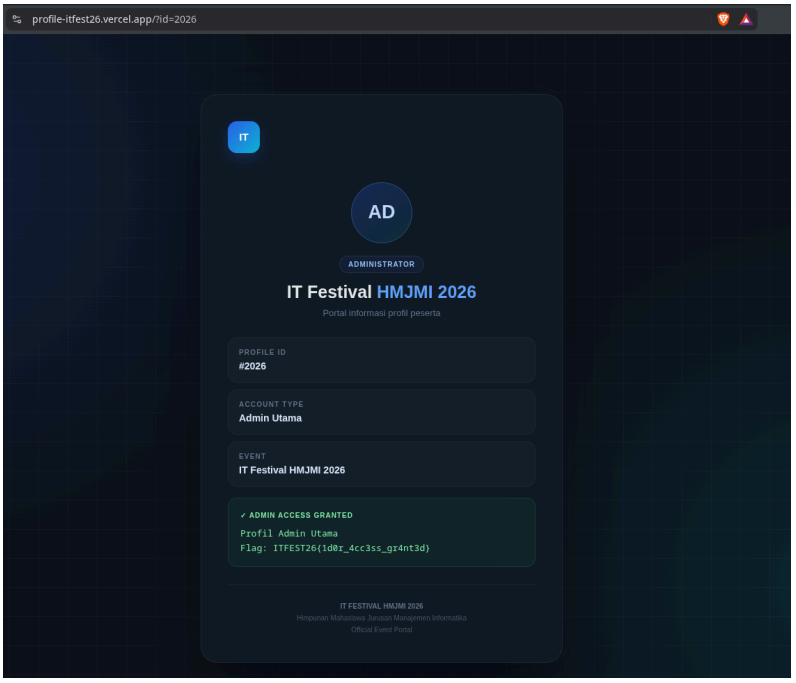
Saat dibuka, website hanya menampilkan profil partisipan, terdapat profile id, tipe akun, dan event.




Saat menambahkan parameter ?id=2 di url, ternyata server mendeteksi perubahan dan profile id menjadi #2




Karena angka 2026 banyak diulang di chall kali ini, dicoba memasukkan value 2026 pada id di url untuk melihat apa yang terjadi.




Saat diubah jadi 2026, akun admin terekspos dan flag berhasil didapat.

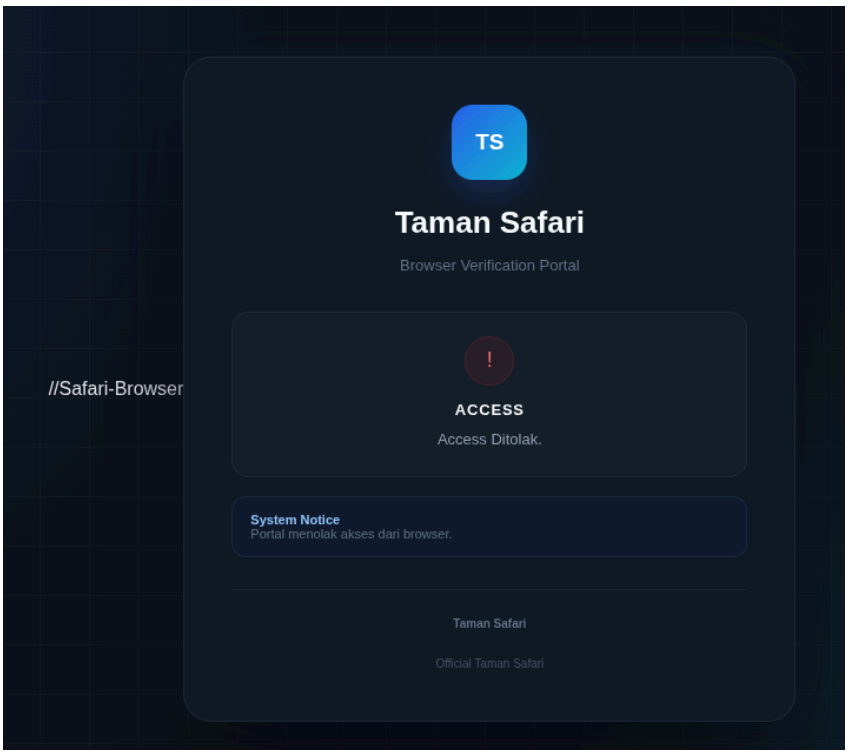
Celah keamanan kali ini adalah IDOR (Insecure Direct Object Reference). Tidak ada lagi authorization check dari sisi server terhadap id apakah memang milik user yang melakukan request atau tidak. Sehingga server percaya begitu saja dan memberikan profil dari id yang diminta saat id diubah.

**ITFEST26{1d0r_4cc3ss_gr4nt3d}**

# Kurir Utusan Rahasia

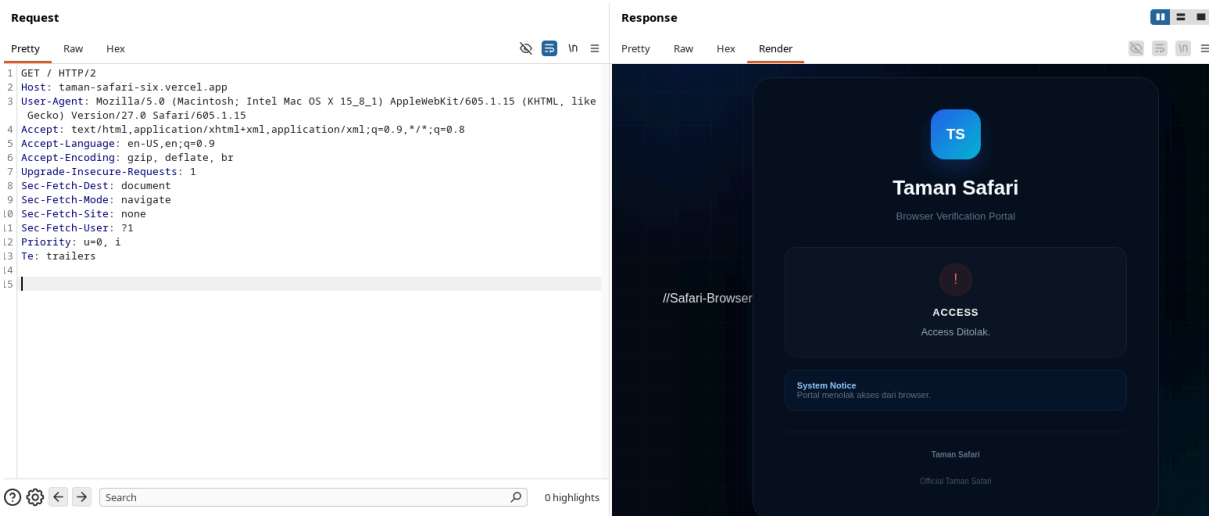
| | |
|---|---|
| **Deskripsi** | Sang pengawal istana, hanya mau membukakan pintu jika pengantar surat mengenakan lencana pengenal resmi kerajaan pada atribut jalurnya. Menyamarlah sebagai utusan yang tepat agar suaramu didengar. |
| **Attachment** | https://taman-safari-six.vercel.app/ |
| **Flag** | **ITFEST26{us3r_4g3nt}** |

Saat website dibuka terdapat sebuah box yang menyatakan akses ditolak, dengan note tambahan “Portal menolak akses dari browser” lalu di kiri ada teks //Safari-Browser.

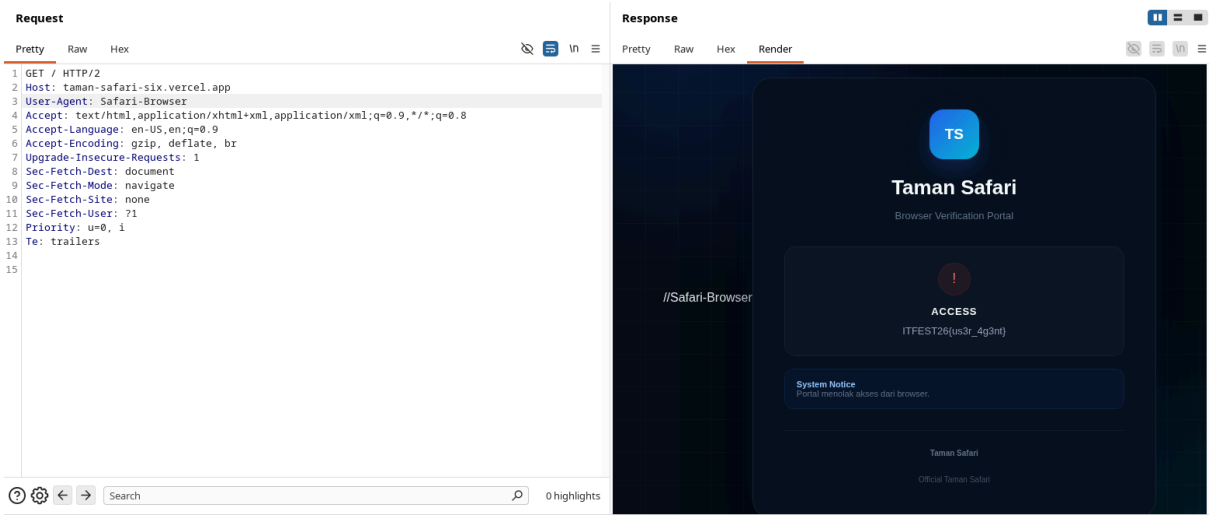



Jika akses browser yang ditolak biasanya website menolak user agent yang sedang digunakan. Dengan teks safari browser ada kemungkinan website hanya menerima user agent dari browser safari.

Namun saat dicoba ganti user agent ke browser safari yaitu **Mozilla/5.0 (Macintosh; Intel Mac OS X 15_8_1) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/27.0 Safari/605.1.15** akses masih ditolak juga.




Karena bingung, akhirnya dicoba secara harfiah user agent nya pakai **Safari-Browser**.




Saat user agent diganti secara harfiah ke Safari-Browser flag langsung keluar dan berhasil didapat.

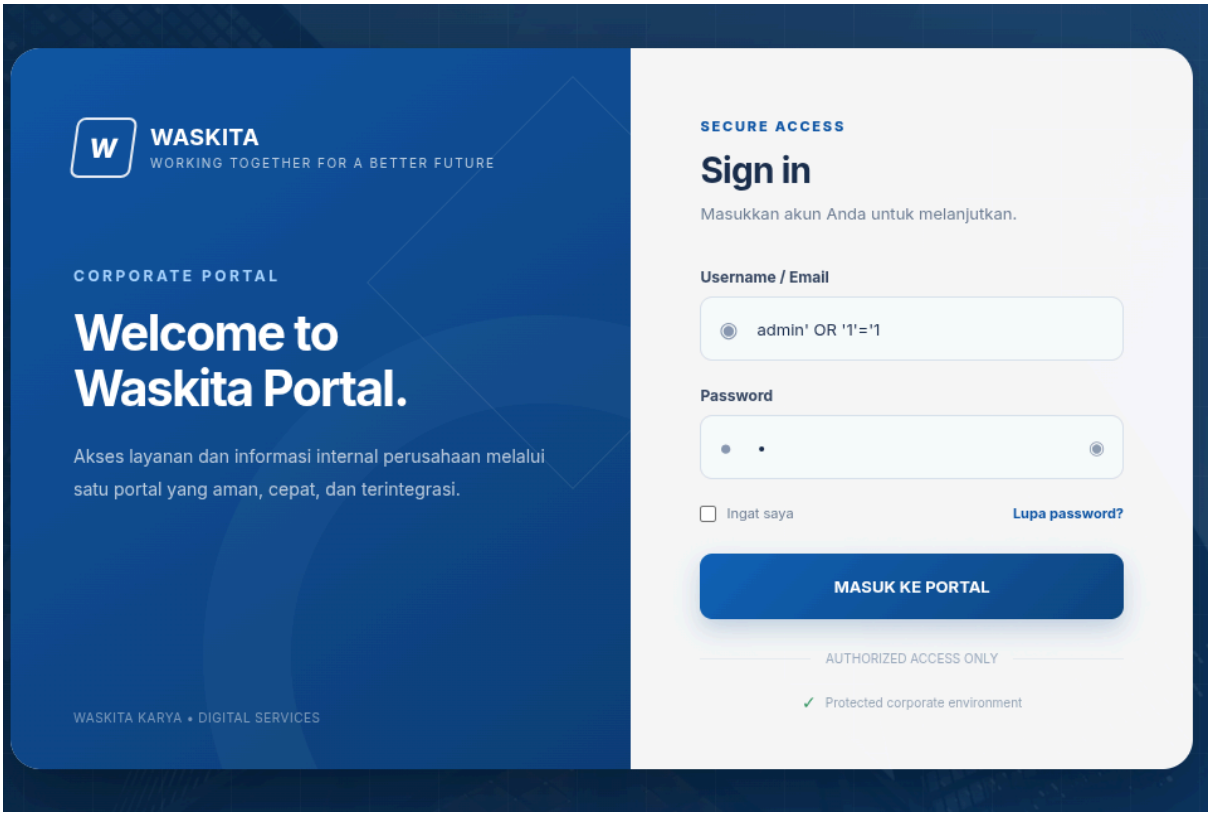
Di sini memang website memblokir akses browser tidak diinginkan, tapi keputusan itu malah bisa dikendalikan oleh user dengan cukup mengganti user-agent pada HTTP Request. Jadi masih sama seperti sebelumnya, yaitu server percaya begitu saja kepada input dari sisi client.

**ITFEST26{us3r_4g3nt}**

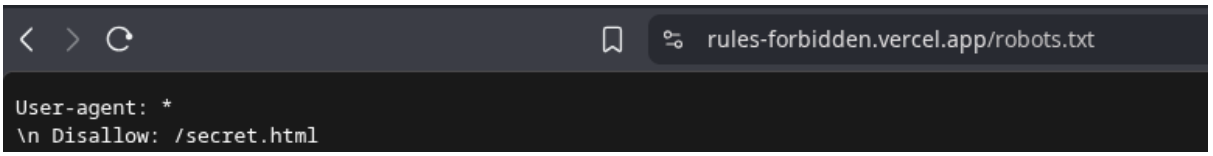
# Kitab Larangan Leluhur

| | |
|---|---|
| **Deskripsi** | Ada lembaran-lembaran yang sengaja disembunyikan dari para pengelana umum agar rahasia tua tidak tersiar ke pasar. Cari petunjuk petunjuk arah yang dilarang dibaca pada gerbang katalog. |
| **Attachment** | https://rules-forbidden.vercel.app/ |
| **Flag** | **ITFEST26{r0b0ts_txt_b0c0r}** |

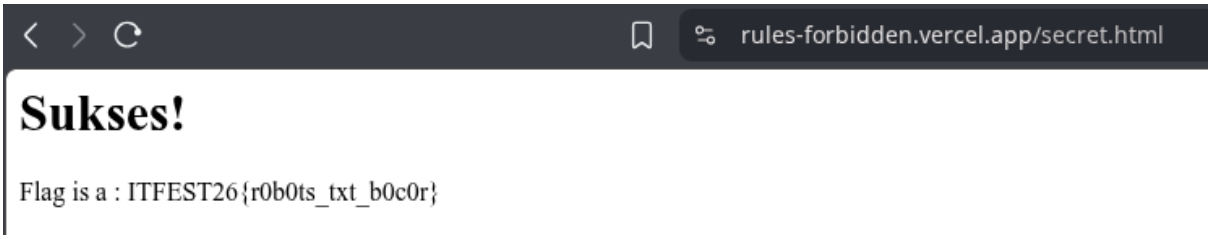
Pada web kali ini terdapat login form lagi. Saat dicoba sql injection lagi ternyata gagal, jadi kali ini celahnya bukan di sql injection




Saat cek source code juga tidak ada yang aneh-aneh. Jadi hal berikutnya yang selalu diperiksa selain source code adalah robots.txt.




Saat melihat robots.txt ada disallow secret.html



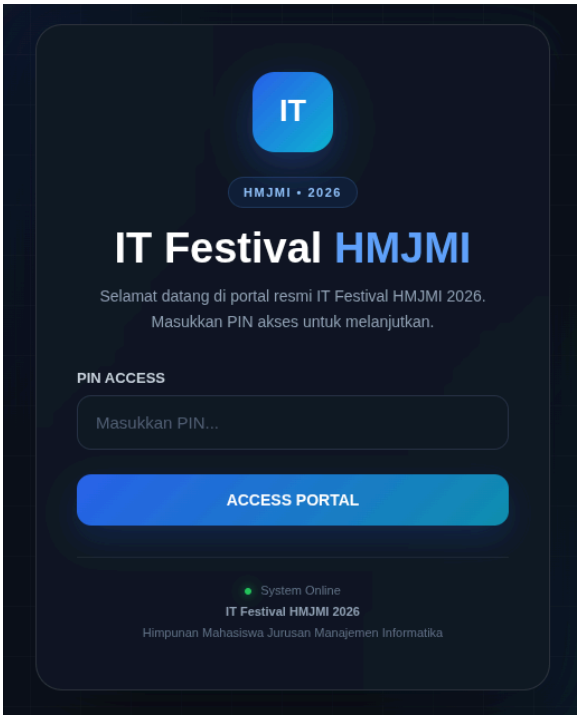

Saat masuk ke secret.html flag-nya didapat. Celah keamanan kali ini hanya sekedar miskonfigurasi biasa di mana informasi penting terekspos di robots.txt dan secret.html tidak ada mekanisme keamanan jadi bisa diakses siapapun.

| **ITFEST26{r0b0ts_txt_b0c0r}** |
|---|

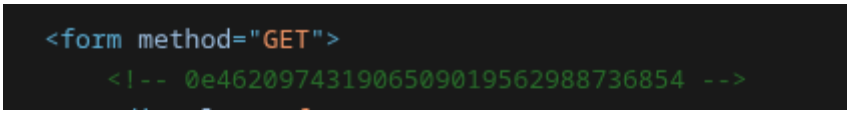
# Sumpah Serapah Angka Nol

| | |
|---|---|
| **Deskripsi** | Para pendeta kuil menggunakan timbangan sakral yang menganggap semua wujud hampa memiliki berat yang setara. Ketika angka misterius berawalan fana dipertemukan dengan timbangan longgar, gerbang rahasia akan bergeser. |
| **Attachment** | https://itfest26.vercel.app |
| **Flag** | **ITFEST26{php_m4g1c_h4sh}** |

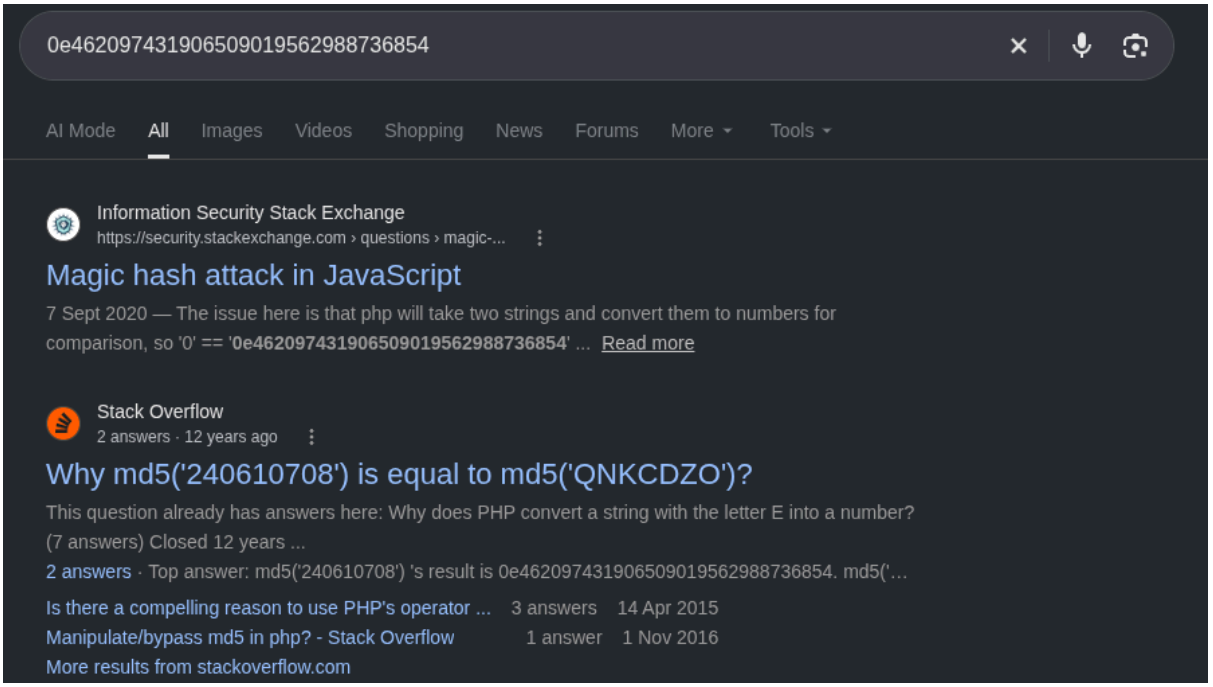
Saat website dibuka terdapat form untuk memasukkan pin.




Saat membuka source code website, terdapat hal menarik ada sebuah teks angka-angka yang dijadikan komentar di source code website.

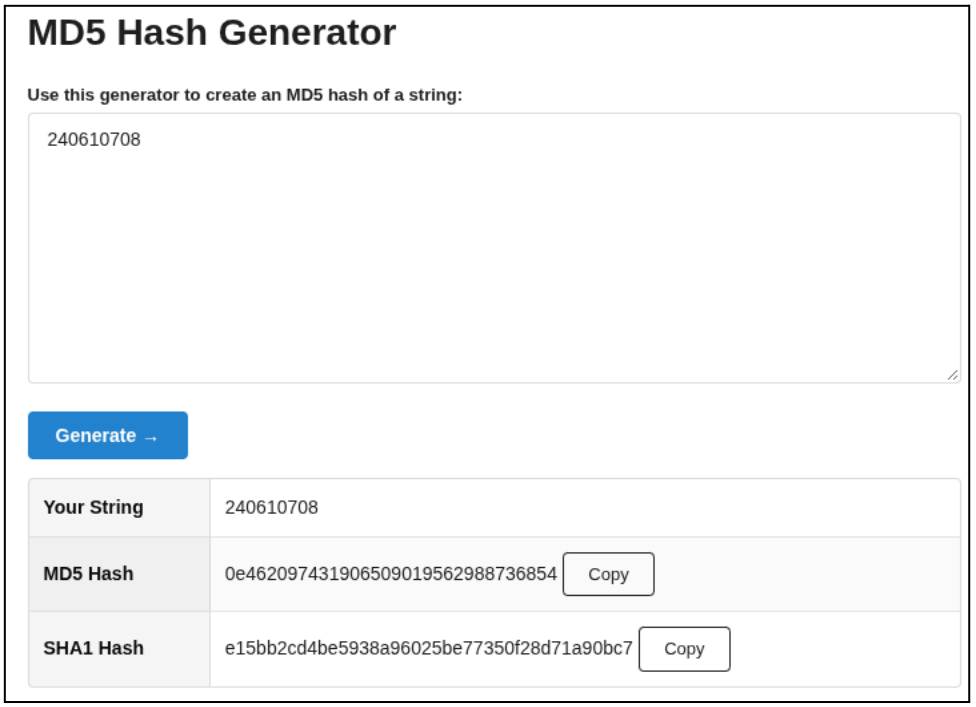



Saat melakukan pencarian di google tentang angka tersebut, hasil pencarian menunjukkan beberapa website yang membahas magic hash attack.



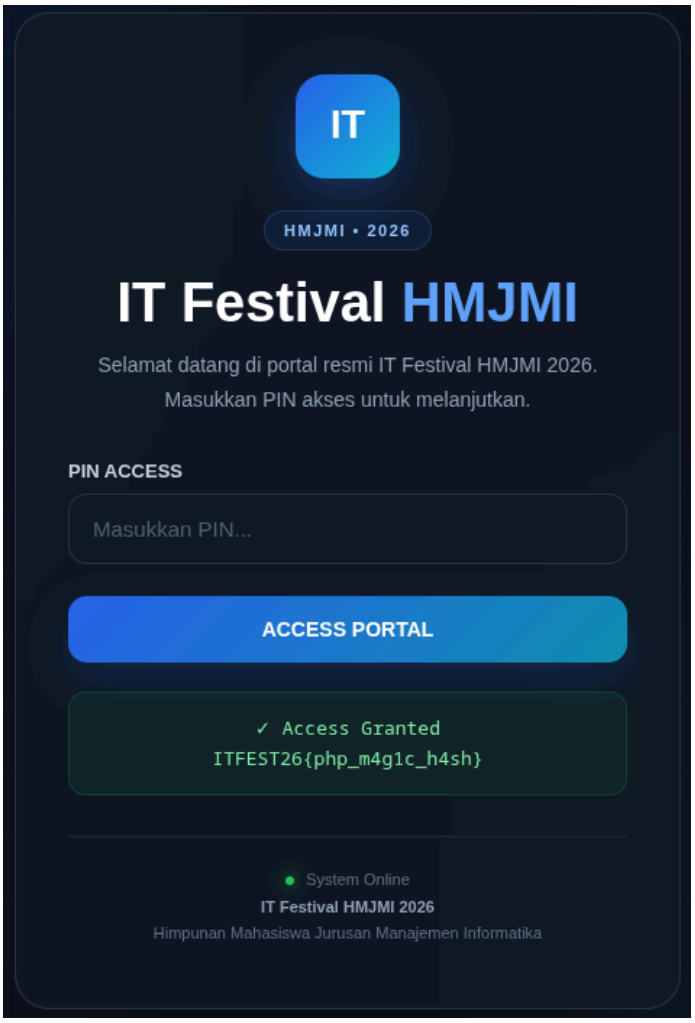

Magic hash attack adalah serangan yang memanfaatkan loose comparison php untuk menghasilkan nilai true dengan menggunakan operator == atau ===.

Saat memasukkan 0e462097431906509019562988736854 sebagai pin hasilnya adalah tidak valid. Tapi dari hasil search google, selain 0e462097431906509019562988736854 juga ada 240610708 yang merupakan nilai asli dari 0e462097431906509019562988736854 sebelum diubah menjadi hash.



Percobaan sebelumnya gagal karena input PIN kemungkinan di-hash dulu oleh sistem sebelum dibandingkan, jadi string magic hash yang dimasukkan ikut di-hash lagi, dan hasilnya bukan magic hash lagi (polanya rusak).

Jadi solusinya bisa dengan memasukkan nilai asli (plaintext) yang kalau di-hash menghasilkan magic hash.



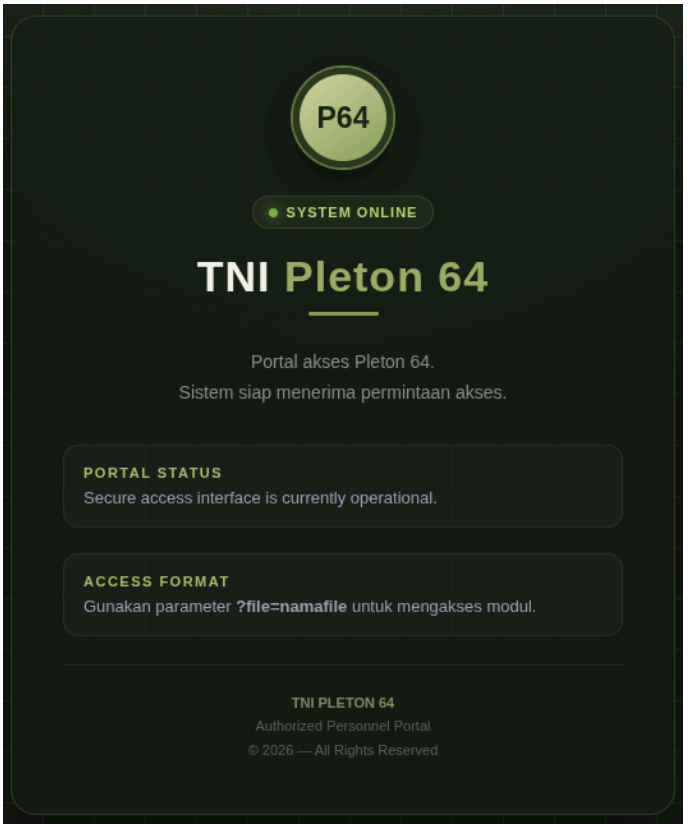

Saat dimasukkan, input diterima dan flag berhasil didapat. Celah keamanan kali ini adalah php loose comparison yang membuat magic hash yang dimasukkan bisa dibandingkan dengan magic hash lain sehingga menghasilkan nilai true. Karena itulah input berhasil diterima.

**ITFEST26{php_m4g1c_h4sh}**

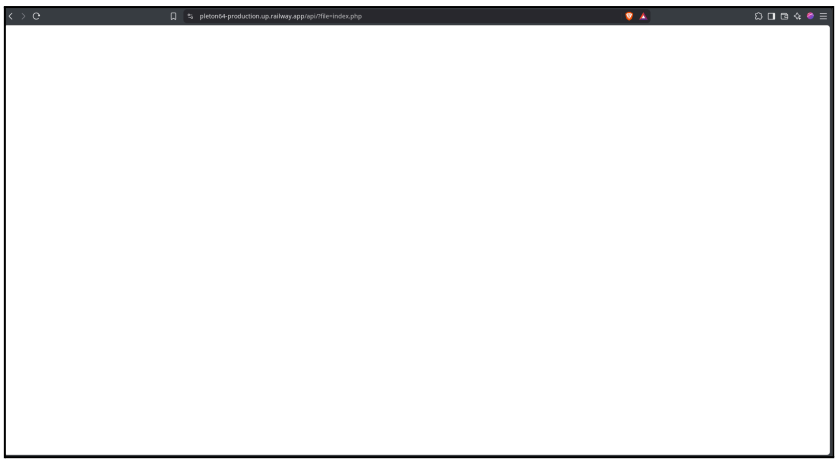
# Labirin Gulungan Sandi

| | |
|---|---|
| **Deskripsi** | Sebuah pesan penting dibungkus berlapis-lapis kain sutra oleh pasukan sandi. Kamu harus mengupas lipatan demi lipatan penutup tersebut secara berurutan hingga tulisan aslinya terang benderang di mata. |
| **Attachment** | https://pleton64-production.up.railway.app/api/ |
| **Flag** | **ITFEST26{lf1_wr4pp3r_php}** |

Saat membuka web terdapat box yang berisi teks tentang sistem pleton64. Salah satu teks menyebutkan gunakan parameter ?file=namafile untuk akses modul.




Namun saat dicoba, hasilnya kosongan saja, tapi dari sini diketahui kalau parameternya bisa digunakan yang mana bisa mencoba satu vulnerability yang memungkinan yaitu Local File Inclusion (LFI).



Karena banyak angka 64 disebutkan juga di chall kali ini, kemungkinan ada keterkaitan dengan base64 juga. Setelah searching tentang local file inclusion dengan base64, ditemukan writeup medium tentang local file inclusion, dengan link berikut ini https://medium.com/@nikomanousos123/advanced-local-file-inclusion-leveraging-php-filters-when-standard-injection-fails-81de4f661f74

Pada writeup medium tersebut ada kasus yang sama yaitu hasilnya kosongan juga saat mencoba parameter. Salah satu payload yang digunakan pada writeup untuk mengatasi hal itu adalah ?file=php://filter/read=convert.base64-encode/resource=index.php, jadi dicoba juga untuk website chall ini. Pada awalnya menghasilkan kosongan juga, tapi setelah dicoba hapus **.php** hasilnya bisa dilihat.




Saat dicoba, berhasil. Isi dari file diberikan dalam bentuk base64, pada contoh adalah index.php yang ketika base64 tersebut di-decode hasilnya adalah source code php untuk halaman utama sebelumnya.

Saat mengganti index menjadi flag atau flag.txt hasilnya kosong. Terus coba-coba lagi kali ini pakai p64 dan pleton64, p64 tidak menghasilkan apa-apa, tapi pleton64 menghasilkan teks base64.



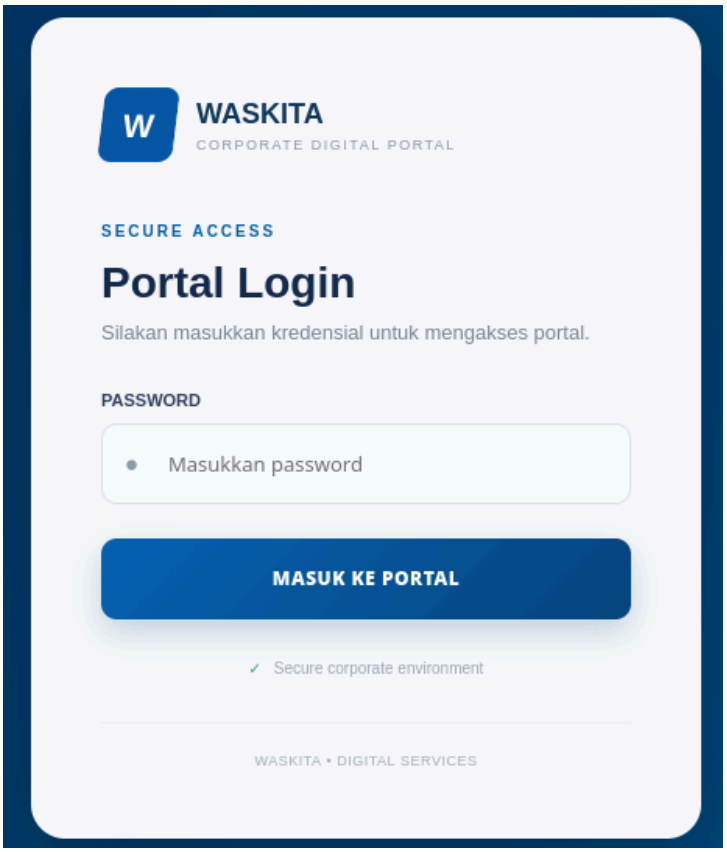

Saat di-decode, base64 tersebut hasilnya <?php //  ITFEST26{lf1_wr4pp3r_php} ?>. Celah keamanan kali ini adalah Local File Inclusion (LFI), di mana parameter file tidak divalidasi di sisi server sehingga bisa disisipkan payload php://filter untuk membaca source code file alih-alih mengeksekusinya.

**ITFEST26{lf1_wr4pp3r_php}**

# Sumur Tanpa Dasar

| | |
|---|---|
| **Deskripsi** | Wadah penampungan air di desa ini tidak memiliki dinding pembatas yang kokoh. Jika kamu menuangkan muatan yang melimpah ruah melampaui takaran normal, dindingnya akan runtuh dan menampakkan isi lantai dasarnya. "H4y0l0h_6im4n4_n1h_m4suk1n_p455ny4" |
| **Attachment** | https://portal-login-one.vercel.app/ |
| **Flag** | **ITFEST26{byp4ss_html_fr0nt3nd}** |

Saat membuka web terdapat halaman login yang hanya meminta password. Saat dicoba dengan memasukkan hal paling mencolok dari deskripsi sebagai password yaitu “H4y0l0h_6im4n4_n1h_m4suk1n_p455ny4” teks terpotong, ternyata maksimal yang bisa diisi di dalam box input hanya 5 karakter.




Jadi berikutnya dicoba intercept dengan burpsuite untuk melihat apakah ada parameter password di dalam request header.

**Request**

Pretty Raw Hex

```
7  Content-Type: application/x-www-form-urlencoded
8  Content-Length: 8
9  Origin: https://portal-login-one.vercel.app
10 Referer: https://portal-login-one.vercel.app/
11 Upgrade-Insecure-Requests: 1
12 Sec-Fetch-Dest: document
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-User: ?1
16 Priority: u=0, i
17 Te: trailers
18
19 pw=H4y0l
```

Ternyata ada parameter password di dalam HTTP request-nya, karena sudah intercept berikutnya tinggal masukkan password full ke dalam parameternya lalu forward.



Setelah di-forward, login berhasil dan flag didapatkan. Celah keamanan kali ini adalah pembatasan karakter password cuma pada frontend, bisa dilihat pada source code website.

```
<div class="input-wrapper">

  <span class="input-icon">●</span>

  <input
    type="text"
    id="pw"
    name="pw"
    maxlength="5"
    placeholder="Masukkan password"
    autocomplete="off"
    required
  >

</div>
```

Parameter yang diterima server tetap tidak memiliki batasan karakter, jadi dengan mengubah parameternya di burpsuite password full akan tetap diterima.

**ITFEST26{byp4ss_html_fr0nt3nd}**

# Cryptography

## Gema dari Dasar Sumur

| | |
|---|---|
| **Deskripsi** | Segel kerajaan ini dikunci menggunakan tiga tetes air garam yang masing-masing diambil dari botol berlabel huruf kecil (a sampai z). Tiga tetes garam tersebut digunakan untuk membuka gembok besi yang mengunci peti Kompi 64. Temukan kombinasi tiga tetes garam yang tepat dengan mencocokkan aroma segel yang tertinggal di bagian luar peti. |
| **Attachment** | crypto_10_tiga_tetes.txtcrypto_10_tiga_tetes.txt |
| **Flag** | ITFest2026{t1g4_t3t3s_g4r4m_p3mbuk4_s3g3l_29d5} |

Dapet file isinya dua nilai: AROMA_SEGEL (32 byte, harusnya sha256) sama PETI_TERKUNCI_HEX (64 byte).

“Tiga tetes garam dari botol a-z” berarti salt 3 huruf kecil, jadi cuma ada 26x3 = 17576 kemungkinan dan bisa langsung di-bruteforce. Ciphertext-nya 64 byte, pas dua kali panjang sha256. Tebakan saya kuncinya sha256(salt) yang diulang 2x terus di-XOR ke ciphertext.




```
import hashlib, itertools, string, base64

C =
bytes.fromhex("0be66d40f05e1ecd58b76515381fad54cb8da770d279c64ac2bc049257
0e07693bda6d73f2351187778a6e5d170b8b02e5a08a2ec778ef56eb92268341645464"
)

for p in itertools.product(string.ascii_lowercase, repeat=3):
    s = "".join(p).encode()
    k = hashlib.sha256(s).digest() * 2
    pt = bytes(a ^ b for a, b in zip(C, k))
    if all(32 <= x < 127 for x in pt):
        print(s, pt, base64.b64decode(pt))
```

**ITFest2026{t1g4_t3t3s_g4r4m_p3mbuk4_s3g3l_29d5}**

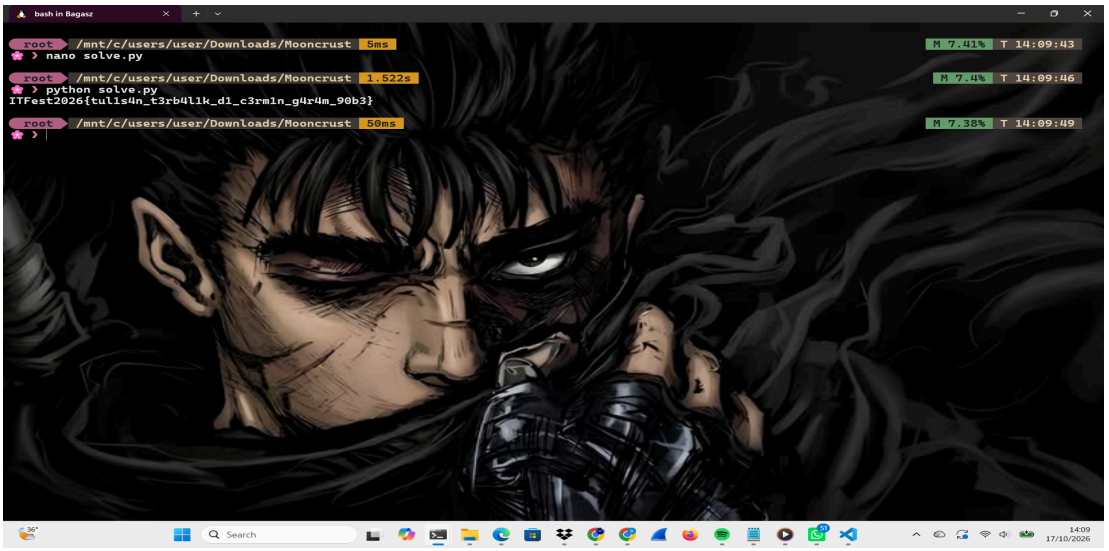
# Kunci Karatan di Saku Jas

| | |
|---|---|
| **Deskripsi** | Kertas perkamen itu dibaca dari ujung kanan ke ujung kiri di depan sebuah cermin cekung. Sebelum dipantulkan ke cermin, setiap pasang huruf telah ditaburi garam secara berselang-seling dengan angka ganjil dan genap, kemudian dilipat mengikuti aturan Kompi 64 sebanyak 4 kali lipatan rapat hingga tak terlihat celah sedikit pun. |
| **Attachment** | crypto_8_cermin_cekung.txt |
| **Flag** | ITFest2026{tul1s4n_t3rb4l1k_d1_c3rm1n_g4r4m_90b3} |

Dari ceritanya: “Kompi 64” itu Base64, “cermin” artinya dibalik, dan “garam ganjil-genap” itu dua kunci yang dipakai gantian.

decode Base64 tiga kali sampai yang keluar tinggal byte acak, lalu urutannya saya balik. Kuncinya tidak diketahui, jadi saya brute force pasangan angka ganjil/genap. Yang menghasilkan flag: byte dikurangi 13 dan 28 secara bergantian

berikut adalah script solve soal



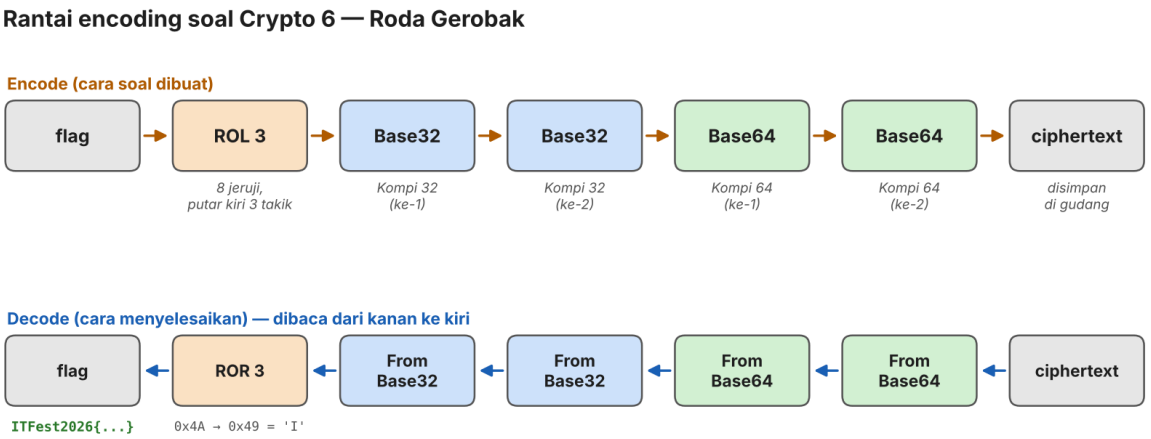

```
import base64
s = open('crypto_8_cermin_cekung.txt').read().strip()
for _ in range(4):
    s = base64.b64decode(s)
s = s[::-1]
print(bytes((x - (13 if i % 2 == 0 else 28)) % 256 for i, x in enumerate(s)).decode())import
base64
s = open('crypto_8_cermin_cekung.txt').read().strip()
for _ in range(4):
    s = base64.b64decode(s)
s = s[::-1]
print(bytes((x - (13 if i % 2 == 0 else 28)) % 256 for i, x in enumerate(s)).decode())
```

**ITFest2026{tul1s4n_t3rb4l1k_d1_c3rm1n_g4r4m_90b3}**

# Langkah Kaki di Atas Daun

| | |
|---|---|
| **Deskripsi** | Gerobak tua itu memiliki roda dengan delapan jeruji besi. Akibat menghantam batu besar, setiap jeruji bergeser berputar ke arah kiri sebanyak 3 takik dari posisi aslinya. Setelah roda diperbaiki seadanya, gerobak tersebut dibawa melintasi markas Kompi 32 sebanyak dua kali, lalu diakhiri dengan mengelilingi lapangan Kompi 64 sebanyak dua kali sebelum akhirnya disimpan di gudang. |
| **Attachment** | crypto_6_roda_gerobak.txt |
| **Flag** | ITFest2026{r0d4_g3r0b4k_b3rput4r_t1g4_g3r1g1_12f0} |

ini adalah contoh alur soal dibuat, dan sampai dapat flagnya




ini hasil flagnya




berikut adalah scriptnya

```
import base64

ct = open("crypto_6_roda_gerobak.txt").read().strip()

d = ct.encode()
for _ in range(2):
    d = base64.b64decode(d)
for _ in range(2):
    d = base64.b32decode(d)
```

```
# 8 jeruji, diputar balik ke kanan 3 takik
flag = bytes(((b >> 3) | (b << 5)) & 0xFF for b in d)
print(flag.decode())import base64

ct = open("crypto_6_roda_gerobak.txt").read().strip()

d = ct.encode()
for _ in range(2):
    d = base64.b64decode(d)
for _ in range(2):
    d = base64.b32decode(d)

flag = bytes(((b >> 3) | (b << 5)) & 0xFF for b in d)
print(flag.decode())
```

**ITFest2026{r0d4_g3r0b4k_b3rput4r_t1g4_g3r1g1_12f0}**

# Teka Teki Kucing Hitam

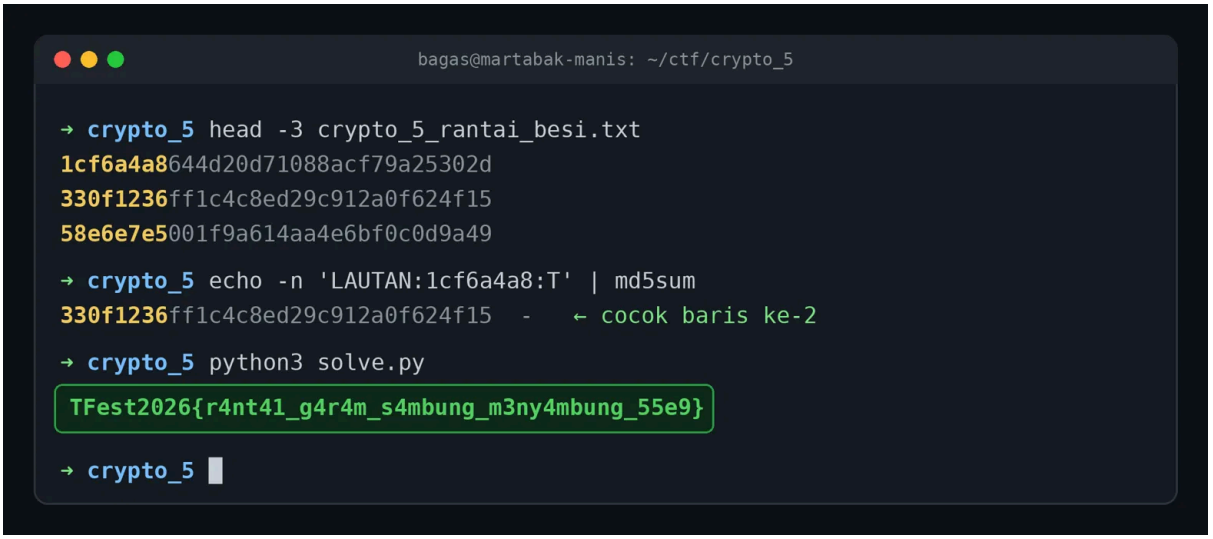
| | |
|---|---|
| **Deskripsi** | Seorang pandai besi menempa rantai baja di mana setiap mata rantai yang baru selalu dicelupkan ke dalam air garam bekas celupan mata rantai tepat di belakangnya. Ia mencampur sejumput pasir pantai bertuliskan 'LAUTAN' bersama satu huruf rahasia dan juga sisa uap dari huruf sebelumnya, lalu meleburnya menjadi abu 32 butir. Kamu harus mengurai mata rantai itu satu demi satu dari ujung paling depan. |
| **Attachment** | crypto_5_rantai_besi.txt |
| **Flag** | ITFest2026{r4nt41_g4r4m_s4mbung_m3ny4mbung_55e9} |

Filenya berisi 48 baris hash MD5. Tiap baris ternyata md5 dari "LAUTAN:" + 8 karakter awal hash sebelumnya (+ ":" +)
satu huruf flag.

Saya sempat mentok karena awalnya pakai hash sebelumnya utuh, baru ketemu setelah sadar soalnya bilang "sisa". Setelah itu tinggal brute-force satu huruf per baris dan flag keluar.

Filenya berisi 48 baris hash MD5. Tiap baris ternyata md5 dari "LAUTAN:" + 8 karakter awal hash sebelumnya + ":" + satu huruf flag.

Saya sempat mentok karena awalnya pakai hash sebelumnya utuh, baru ketemu setelah sadar soalnya bilang "sisa". Setelah itu tinggal brute-force satu huruf per baris dan flag keluar.




oke kita berhasil mendapatkan flagnya berikut adalah script solve, disini saya menambahkan ITFEST{} karena sesuai format soal

```
import hashlib
```

```
L = [l.strip() for l in open('crypto_5_rantai_besi.txt') if l.strip()]
md5 = lambda s: hashlib.md5(s.encode()).hexdigest()

flag = ''
for i in range(1, len(L)):
    for c in map(chr, range(32, 127)):
        if md5(f"LAUTAN:{L[i-1][:8]}:{c}") == L[i]:
            flag += c
            break
print(flag)
```

**ITFest2026{r4nt41_g4r4m_s4mbung_m3ny4mbung_55e9}**

# Surat yang Tak Pernah Sampai

| | |
|---|---|
| **Deskripsi** | Seorang kurir kerajaan diperintahkan berlari mengelilingi benteng Kompi 64 sebanyak 5 kali putaran penuh tanpa henti. Setiap kali ia merasa lelah di tengah jalan, ia memutar jarum arlojinya maju sebanyak 13 angka agar waktu terasa lebih cepat. Sesampainya di gerbang utama, ia harus mencocokkan detak jantungnya secara berulang-ulang dengan ketukan pedang penjaga bernama 'GARUDA' agar pintu ruang rahasia terbuka. |
| **Attachment** | crypto_2_kurir_benteng.txt |
| **Flag** | ITFest2026{p3rj4l4n4n_l1m4_k4l1_m3m4nJ4t_d1nd1ng_77b2} |

Filenya berisi satu baris panjang yang sekilas mirip Base64, tapi kalau langsung di-decode hasilnya rusak. Ternyata ceritanya adalah urutan lapisannya: "maju 13 angka" itu ROT13, "Kompi 64 sebanyak 5 kali putaran" itu Base64 yang di-decode 5 kali, dan "mencocokkan berulang-ulang dengan 'GARUDA'" itu repeating-key XOR dengan kunci GARUDA.

Sempat mentok karena lapisan terakhir Vigenère, padahal setelah Base64 datanya sudah byte mentah, bukan huruf. Setelah diganti XOR, flag langsung keluar.

berikut script solve




```
import base64, codecs

c = open('crypto_2_kurir_benteng.txt').read().strip()

s = codecs.encode(c, 'rot13').encode()

for _ in range(5):
    s = base64.b64decode(s)

k = b'GARUDA'
print(bytes(b ^ k[i % len(k)] for i, b in enumerate(s)).decode())
```

**ITFest2026{p3rj4l4n4n_l1m4_k4l1_m3m4nJ4t_d1nd1ng_77b2}**

# Parade Bayangan Semu

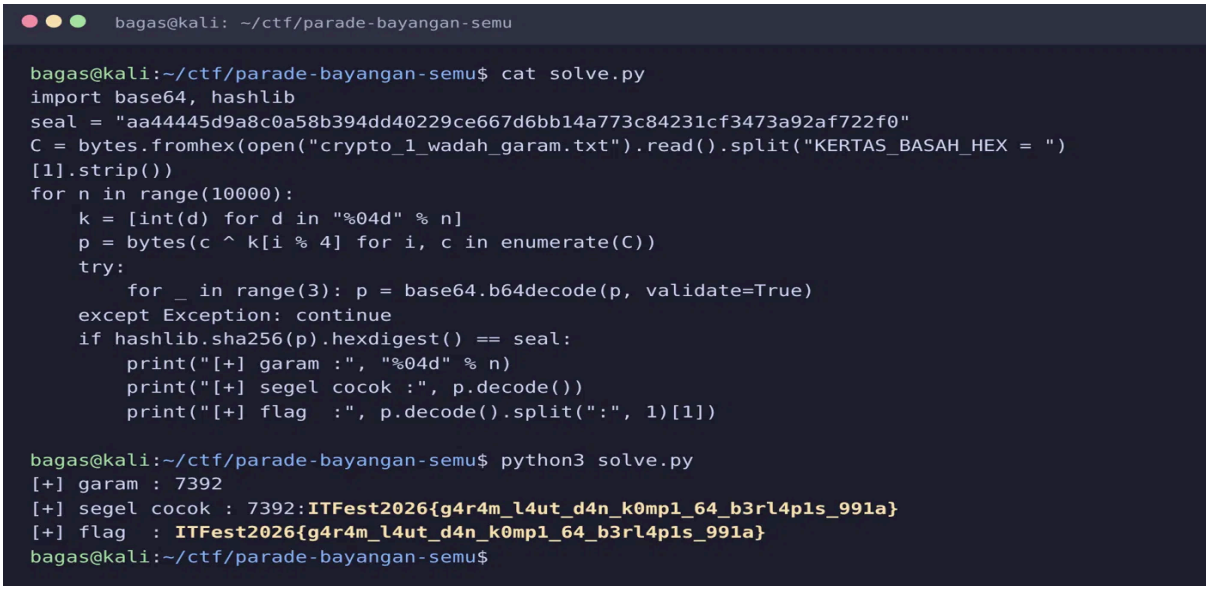
| | |
|---|---|
| **Deskripsi** | Sebuah wadah kayu tua ditemukan terdampar di tepi pantai setelah dicuci bersih menggunakan air garam laut murni yang diambil dari empat penjuru mata angin (0000-9999). Di dalamnya terdapat gulungan kertas yang telah basah oleh larutan garam tersebut. Kertas itu kemudian dibawa oleh Kompi 64 yang hendak berangkat berperang. Sebelum membuka segelnya, komandan menyuruh seluruh pasukan mengelilingi markas Kompi 64 sebanyak 3 kali berturut-turut sambil menyanyikan sandi bayangan. |
| **Attachment** | crypto_1_wadah_garam.txt |
| **Flag** | ITFest2026{g4r4m_l4ut_d4n_k0mp1_64_b3rl4p1s_991a |

Isi filenya cuma dua: hash SHA256 yang jadi segel, sama ciphertext dalam bentuk hex. Waktu hex-nya saya decode, hasilnya kelihatan kayak base64, tapi ada beberapa karakter yang nyasar seperti <, [, dan }.

Dari situ teksnya di-XOR pakai kunci 4 digit, soalnya di soal disebut garam dari 0000 sampai 9999. Awalnya saya sempat mentok karena kuncinya saya XOR sebagai teks ("0000" sampai "9999"), dan nggak ada satu pun yang cocok.

Ternyata yang dipakai itu angkanya langsung, bukan teksnya. Terus bagian "mengelilingi markas Kompi 64 sebanyak 3 kali" ternyata maksudnya base64 yang di-decode tiga kali. Jadi tinggal brute-force semua kunci dari 0000 sampai 9999: XOR, decode base64 tiga kali, lalu cek SHA256-nya sama nggak dengan segel. Kuncinya ketemu di 7392, dan flag-nya keluar.

berikut script auto solvenya




berikut script solvenya :

```
import base64, hashlib
seal = "aa44445d9a8c0a58b394dd40229ce667d6bb14a773c84231cf3473a92af722f0"
C                                                                                =
bytes.fromhex(open("crypto_1_wadah_garam.txt").read().split("KERTAS_BASAH_HEX =
")[1].strip())
for n in range(10000):
    k = [int(d) for d in "%04d" % n]
    p = bytes(c ^ k[i % 4] for i, c in enumerate(C))
    try:
        for _ in range(3): p = base64.b64decode(p, validate=True)
    except Exception: continue
    if hashlib.sha256(p).hexdigest() == seal:
        print("[+] garam :", "%04d" % n)
        print("[+] segel cocok :", p.decode())
        print("[+] flag  :", p.decode().split(":", 1)[1])
```

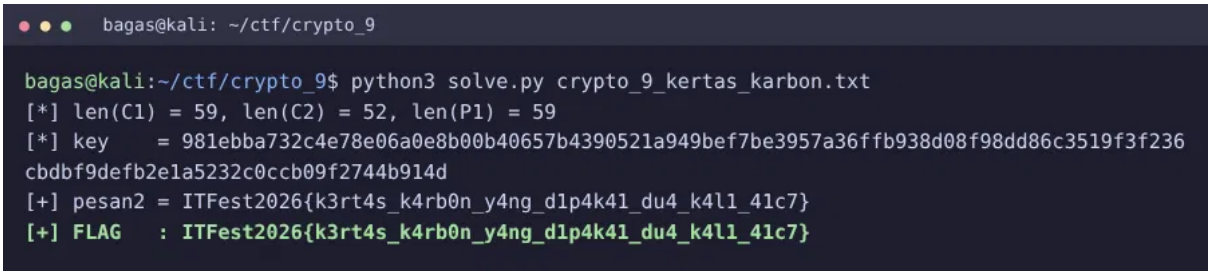
**ITFest2026{g4r4m_l4ut_d4n_k0mp1_64_b3rl4p1s_991a**

# benang Merah yang Kusut

| | |
|---|---|
| **Deskripsi** | Seorang juru tulis yang pelit menggunakan selembar kertas karbon yang persis sama untuk menyalin dua pucuk surat rahasia yang berbeda. Surat pertama diketahui berisi pesan pengantar 'PERINGATAN_DARURAT_UNTUK_SELURUH_PASUKAN_GARIS_DEPAN_SEGERA', sedangkan surat kedua berisi lokasi persembunyian harta karun. Kedua hasil tindasan karbon tersebut kini jatuh ke tanganmu. |
| **Attachment** | crypto_9_kertas_karbon.txt |
| **Flag** | ITFest2026{k3rt4s_k4rb0n_y4ng_d1p4k41_du4_k4l1_41c7} |

Filenya berisi dua ciphertext dalam hex. Petunjuknya ada di cerita: "kertas karbon yang sama untuk dua surat" itu artinya kunci XOR yang sama dipakai dua kali (two-time pad).

Karena isi surat pertama sudah dikasih, kuncinya tinggal dicari dengan K = C1  P1, lalu surat kedua dibuka dengan P2 = C2 K. C2 lebih pendek dari kuncinya, jadi seluruh isinya langsung terbaca dan isinya flag.




```
import re
import sys

KNOWN = b"PERINGATAN_DARURAT_UNTUK_SELURUH_PASUKAN_GARIS_DEPAN_SEGERA"


def xor(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))


def main():
    path = sys.argv[1] if len(sys.argv) > 1 else "crypto_9_kertas_karbon.txt"
    data = open(path).read()

    c1 = bytes.fromhex(re.search(r"TINDASAN_PERTAMA_HEX\s*=\s*([0-9a-fA-F]+)", data).group(1))
    c2 = bytes.fromhex(re.search(r"TINDASAN_KEDUA_HEX\s*=\s*([0-9a-fA-F]+)", data).group(1))

    print(f"[*] len(C1) = {len(c1)}, len(C2) = {len(c2)}, len(P1) = {len(KNOWN)}")

    # K = C1 xor P1
    key = xor(c1, KNOWN)
    print(f"[*] key    = {key.hex()}")

    # P2 = C2 xor K
    p2 = xor(c2, key)
    print(f"[+] pesan2 = {p2.decode()}")

    flag = re.search(rb"[A-Za-z0-9_]+\{[^}]+\}", p2)
    if flag:
        print(f"[+] FLAG   : {flag.group().decode()}")
```

```
if __name__ == "__main__":
    main()
```

**ITFest2026{k3rt4s_k4rb0n_y4ng_d1p4k41_du4_k4l1_41c7}**

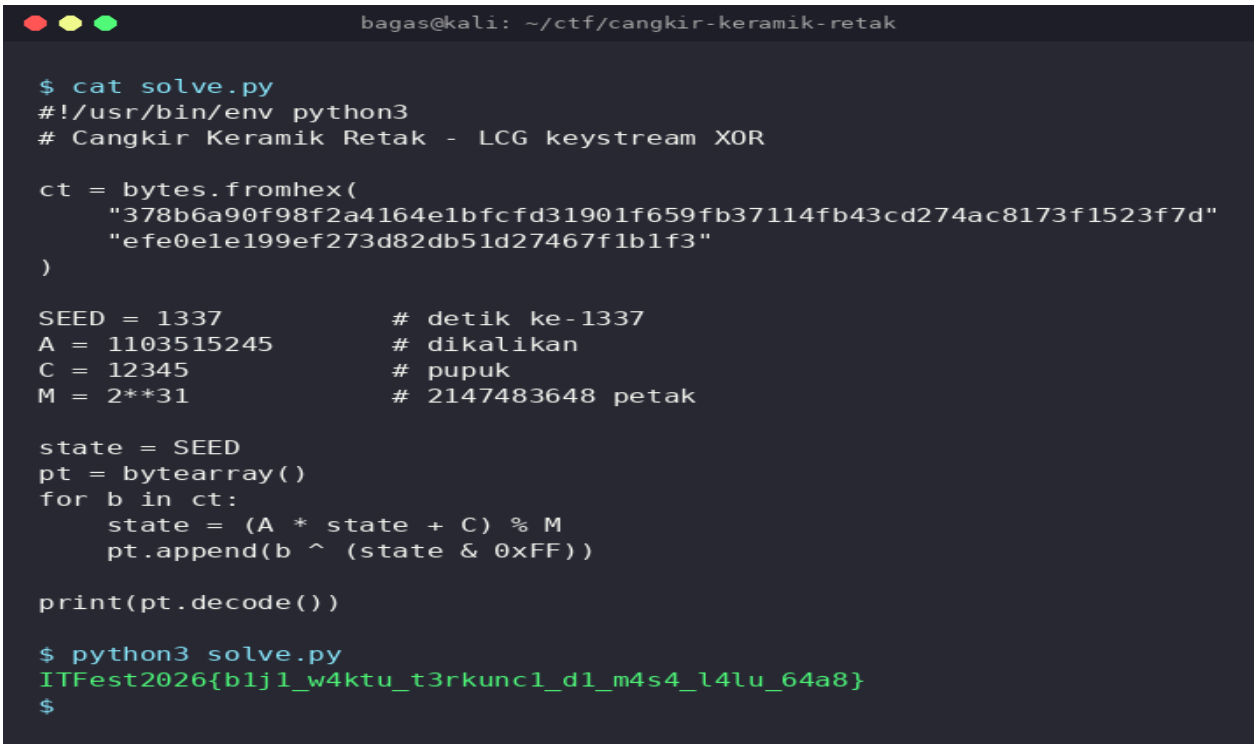
# Cangkir Keramik Retak

| | |
|---|---|
| **Deskripsi** | Sang petani menanam benih pohon ajaib tepat pada detik ke-1337 sejak matahari terbit. Setiap detiknya, tinggi pohon itu dikalikan dengan 1103515245 lalu ditambah pupuk sebanyak 12345 gram di dalam kebun berukuran 2147483648 petak. Daun-daun yang gugur dari pohon itulah yang menutupi tulisan di batu nisan di bawahnya. Hati-hati terhadap daun palsu yang sengaja diletakkan untuk menipu mata mesin. |
| **Attachment** | crypto_7_pohon_waktu |
| **Flag** | ITFest2026{b1j1_w4ktu_t3rkunc1_d1_m4s4_l4lu_64a8} |

Filenya berisi satu ciphertext dalam hex (BATU_NISAN). Petunjuknya ada di cerita: benih ditanam pada detik ke-1337, lalu setiap detik tingginya dikalikan 1103515245, ditambah pupuk 12345, di kebun berukuran 2147483648 petak.

Itu adalah rumus LCG (Linear Congruential Generator) state = (a * state + c) mod m dengan seed = 1337, a = 1103515245, c = 12345, dan m = 2x31. "Daun-daun yang gugur" adalah keystream yang menutupi tulisan di batu nisan, jadi ciphertext di-XOR dengan keluaran LCG. Byte kunci diambil dari byte terendah setiap state (state & 0xff), mulai dari state hasil iterasi pertama (seed sendiri tidak dipakai).

Plaintext didapat dengan P = C ⊕ K. Peringatan soal "daun palsu" adalah jebakan untuk cara ekstraksi lain, misalnya >>16 gaya rand() glibc atau ikut memakai seed sebagai byte kunci pertama. Cara-cara itu menghasilkan sampah atau byte pertama yang rusak, sedangkan & 0xff langsung memberi flag.



```
ct = bytes.fromhex(
   "378b6a90f98f2a4164e1bfcfd31901f659fb37114fb43cd274ac8173f1523f7d"
   "efe0e1e199ef273d82db51d27467f1b1f3"
)

SEED = 1337         # detik ke-1337
A = 1103515245      # dikalikan
C = 12345           # pupuk
M = 2**31           # 2147483648 petak

state = SEED
pt = bytearray()
for b in ct:
   state = (A * state + C) % M
   pt.append(b ^ (state & 0xFF))

print(pt.decode())
```

**ITFest2026{b1j1_w4ktu_t3rkunc1_d1_m4s4_l4lu_64a8}**

# Nyanyian Burung Besi

| | |
|---|---|
| **Deskripsi** | Terdapat sebuah ruangan bundar dengan 256 pintu bernomor urut. Setiap kali seseorang masuk melalui pintu tertentu, lantai ruangan akan berputar mengalikan nomor pintu tersebut dengan 19 lalu menambahkan 47 butir garam laut ke atas timbangan mekanis. Jika beratnya melebihi jumlah pintu yang ada, timbangan hanya akan mencatat sisa buangannya saja. Temukan urutan pintu awal sebelum lantai ruangan ini diputar. |
| **Attachment** | crypto_4_ruang_bundar.txt |
| **Flag** | ITFest2026{c3rm1n_l4b1r1n_m0dul0_du4_l1m4_3n4m_88d1} |

Filenya berisi 52 angka. Ceritanya sebenarnya menjelaskan cara angka-angka itu dibuat:

256 pintu berarti angkanya selalu 0–255. Kalau lebih, yang diambil cuma sisa baginya (modulo 256).
Dikali 19 lalu ditambah 47" adalah rumus enkripsinya.

Jadi rumusnya C = (19 × P + 47) mod 256. Ini disebut affine cipher.

Untuk mengembalikannya, kita jalankan rumusnya terbalik: kurangi 47 dulu, lalu "bagi" dengan 19. Masalahnya, dalam modulo kita tidak bisa membagi biasa, jadi kita kalikan dengan kebalikannya (invers). Kebalikan 19 di modulo 256 adalah 27, karena 19 × 27 = 513, dan 513 dibagi 256 sisanya 1.

```
bagaskara@kali: ~/ctf/ruang_bundar

bagaskara@kali:~/ctf/ruang_bundar$ python3 solve.py
[*] Panjang ciphertext : 52
[*] Invers 19 mod 256  : 27  (cek: 19*27 mod 256 = 1)
[+] Flag : ITFest2026{c3rm1n_l4b1r1n_m0dul0_du4_l1m4_3n4m_88d1}
bagaskara@kali:~/ctf/ruang_bundar$
```

```
import re

A, B, M = 19, 47, 256

with open("crypto_4_ruang_bundar.txt") as f:
   data = f.read()
ct = list(map(int, re.findall(r"\d+", data)))

a_inv = pow(A, -1, M)  # invers modular 19 mod 256 = 27
print(f"[*] Panjang ciphertext : {len(ct)}")
print(f"[*] Invers {A} mod {M}  : {a_inv}  (cek: {A}*{a_inv} mod {M} = {A * a_inv % M})")

pt = bytes((a_inv * (c - B)) % M for c in ct)
```

```
print(f"[+] Flag : {pt.decode()}")
```

**ITFest2026{c3rm1n_l4b1r1n_m0dul0_du4_l1m4_3n4m_88d1}**

# Peta Menuju Pulau Khayalan

| | |
|---|---|
| **Deskripsi** | Di sebuah jembatan gantung yang dijaga oleh 16 prajurit kembar, setiap langkah kaki yang kamu ambil akan menambah beban jembatan tepat sebesar nomor urut langkahmu sendiri dikalikan tujuh. Jangan pernah percaya pada bisikan suara buatan yang menyuruhmu melompat langsung ke ujung jembatan, karena papan-papan kayu di sana telah dipasangi perangkap ilusi bagi mereka yang malas menghitung satu per satu dari pijakan pertama. |
| **Attachment** | crypto_3_jembatan.txt |
| **Flag** | ITFest2026{l4angk4h_k4k1_b3rub4h_s3t14p_m3t3r_33c4} |

File ini berisi 51 byte dalam bentuk hex. Cerita di soal sebenarnya menjelaskan cara byte-byte itu dibuat:

16 prajurit kembar berarti datanya ditulis dalam heksadesimal (basis 16).
Setiap langkah menambah beban sebesar nomor urut langkah × 7 rumus kuncinya.
Setiap byte punya kunci sendiri, yaitu nomor langkah × 7.

Hitung satu per satu dari pijakan pertama  berarti langkah dihitung mulai dari 1, bukan 0.
Bisikan suara buatan … perangkap ilusi" adalah jebakan. Kalau mulai menghitung dari 0 atau mencoba satu kunci untuk semua byte, hasilnya akan berantakan

```
bagas@kali: ~/ctf/itfest2026/jembatan

bagas@kali:~/ctf/itfest2026/jembatan$ python3 solve.py crypto_3_jembatan.txt
[+] Ciphertext : 51 byte
[-] Mulai dari 0 (perangkap) : b'N]]lL})95Opu;Xe~l=\x93V'...
[*] Contoh beberapa langkah pertama:
    langkah  1: 0x4e ^   7 = 0x49 -> 'I'
    langkah  2: 0x5a ^  14 = 0x54 -> 'T'
    langkah  3: 0x53 ^  21 = 0x46 -> 'F'
    langkah  4: 0x79 ^  28 = 0x65 -> 'e'
    langkah  5: 0x50 ^  35 = 0x73 -> 's'
[+] FLAG : ITFest2026{l4angk4h_k4k1_b3rub4h_s3t14p_m3t3r_33c4}
bagas@kali:~/ctf/itfest2026/jembatan$
```

ini script solve

```
c                                                
```

**ITFest2026{l4angk4h_k4k1_b3rub4h_s3t14p_m3t3r_33c4}**